

AMAP - Product management

TECHNICAL REPORT - SPRINT 2 - VERSÃO 1.0

[1090470 - BRUNO SANTOS]

[1190640 - GUILHERME COTA]

[1230213 - TOMÁS OLIVEIRA]

[1230196 - LUÍS SERAPICOS]

[1190007 - HELLEN SANTOS]

1. Introduction.....	6
1.1 Purpose.....	6
1.2 Meetings.....	7
1.3 Convention.....	8
2. Technology.....	9
3. Component Diagram	10
4. Domain Model Diagram	11
5. System Features	14
5.1 US01	14
5.1.1 Story Mapping.....	15
5.1.2 Development and Prioritization	16
5.1.3 Process View	17
5.2 US02	19
5.2.1 Story Mapping.....	20
5.2.2 Development and Prioritization	21
5.2.3 Process View	22
5.3 US03	24
5.3.1 Story Mapping.....	24
5.3.2 Development and Prioritization	25
5.3.4 Process View	27
5.4 US04	31
5.4.1 Story Mapping.....	32
5.4.2 Development and Prioritization	33
5.4.3 Process View	34
5.5 US05	38
5.5.1 Story Mapping.....	39
5.5.2 Development and Prioritization	40
5.5.3 Process View	41
5.6 US06	43
5.6.1 Story Mapping.....	43
5.6.2 Development and Prioritization	44
5.6.3 Process View	45
5.7 US07	47
5.7.1 Story Mapping.....	47

5.7.2 Development and Prioritization	48
5.7.3 Process View	49
5.8 US08	52
5.8.1 Story Mapping.....	52
5.8.2 Development and Prioritization	53
5.8.3 Process View	54
5.9 US09	55
5.9.1 Story Mapping.....	56
5.9.2 Development and Prioritization	57
5.9.3 Process View	59
6. Backbone	62
6.1 Frontend	63
6.1.1 Middleware	64
6.2 Public Pages	65
6.2.1 Login Page	65
6.2.2 Register Page	67
6.2.3 Reset Page.....	68
6.2.4 Confirm	69
6.3 Backend	71
6.3.1 Middleware	72
6.3.2 Controllers / Services.....	73
6.3.3 Logger	75
6.3.4 Routes.....	76
6.4 Database	77
6.4.1 Seeders and Migrations	81
6.4.2 Database Triggers	82
7. Non-functional requirements.....	84

Figure 1 - Project repository.....	8
Figure 2 - Component diagram	10
Figure 3 -Model diagram.....	11
Figure 4 - Story mapping US1	15
Figure 5 - Jira User Stories.....	16
Figure 6 - Dashboard view.....	17
Figure 7 - Amap catalog view.....	17
Figure 8 - Register product view	18
Figure 9 - Story mapping US 2	20
Figure 10 - Jira User Stories	21
Figure 11 - Jira User Stories	21
Figure 12 - Jira User Stories	22
Figure 13 - Dashboard view	22
Figure 14 - Amap management view.....	22
Figure 15 - Season configs view	23
Figure 16 - Season selection view.....	23
Figure 17 - Story mapping US3.....	24
Figure 18 - Jira User Stories	25
Figure 19 - Jira User Stories	26
Figure 20 - Jira User Stories	26
Figure 21 - Amap catalog view	27
Figure 22 - Register product	28
Figure 23 - Producer profile view	29
Figure 24 - Product edit view.....	30
Figure 25 - Story mapping US04	32
Figure 26 - Jira User Stories	34
Figure 27 - Jira User Stories	34
Figure 28 - Dashboard view	34
Figure 29 - Amap catalog view	35
Figure 30 - Product details view	35
Figure 31 - Subscripition view	36
Figure 32 - Add to cart view	36
Figure 33 - Cart view.....	37
Figure 34 - Quantity option view.....	37
Figure 35 - Payment view	37
Figure 36 -Story mapping US05	39
Figure 37 - Jira User Stories	40
Figure 38 - Jira User Stories	40
Figure 39 - Jira User Stories	41
Figure 40 - Menu view	41
Figure 41 - Amounts coproducers view.....	42
Figure 42 - Amounts producers view.....	42
Figure 43 - Story Mapping for the user story US06: "producer should be able to update their orders quantities based on the production changes".	43
Figure 44 - Jira User Stories	44
Figure 45 - This view "Producers Orders" shows the orders made to the products that the current Producer is able to produce.	45
Figure 46 - The input validation only allows the user to update the quantity to non-negative values.	45

Figure 47 - Once the user triggers the "Update Order quantities" button the order quantity is updated with the correct value.	46
Figure 48 - Story mapping US07	47
Figure 49 - Jira User Stories.	48
Figure 50 - Jira User Stories.	48
Figure 51 - Jira User Stories.	49
Figure 52 - Amap management view.....	49
Figure 53 - KPIs menu view.	50
Figure 54 - KPIs view.	50
Figure 55 - KPIs view.	51
Figure 56 - Story Mapping for the user story US08: "AMAP Managers should be able to trigger paid values and delivery values calculations and place the difference on co-producer's accounts"	52
Figure 57 - Jira User Stories.	53
Figure 58 - Co-producer profile view where they can, beside other features, see the values that they need to pay/receive from pending orders.	54
Figure 59 - Story mapping US09	56
Figure 60 - Jira User Stories.	58
Figure 61 - Jira User Stories.	59
Figure 62 - Amap catalog view.	59
Figure 63 - Create basket view.	60
Figure 64 - Product and products producer profile view.	61
Figure 65 - FE structure	63
Figure 67- Login page	65
Figure 68 - Reset password page	66
Figure 69 - Register page.....	67
Figure 70 - Reset page.....	68
Figure 71 - Email verification.....	69
Figure 72 - BE structure	71
Figure 73 - Middleware structure	72
Figure 74 - Controllers/service structure	73
Figure 75 - Utils structure.....	75
Figure 76 - Routes structure	76
Figure 77 - Swagger endpoint example.....	76
Figure 78 - Database tables.....	77
Figure 79 - Cart view.....	85
Figure 80 UI screenshot of the application's navbar.....	87
Figure 81 UI screenshot of a producer's profile page.....	88

1. Introduction

AMAP (Associação pela Manutenção da Agricultura de Proximidade) or CSA (CSA - Comunidade que Sustenta a Agricultura) play an essential role in connecting producers and consumers of organic products, promoting sustainable practices and healthier eating.

This report presents the start of the application development process, which is based on the requirements gathering carried out previously and documented in the SRS (Software Requirements Specification). The current phase is dedicated to implementation, where the identified requirements are translated into concrete functionalities, taking into account the operational particularities of AMAPs and good software engineering practices.

This document details the technical progress made, as well as the strategies and tools adopted, providing a clear overview of the work carried out to date and the next steps towards completing the project.

1.1 Purpose

The purpose of this project is to develop an intuitive and efficient application that supports the management of AMAPs, promoting sustainability and the local economy. The solution aims to create a digital platform that facilitates the registration of producers and co-producers, the management of organic products and their involvement.

By offering features such as order organization and transparency in transactions, the application aims to help responsible consumption and access to quality organic food. It will also simplify current traditional processes, allowing for greater efficiency and operational simplification.

1.2 Meetings

As part of the application's development, sprint meetings played a central role in ensuring team coordination and compliance with the established objectives. These meetings were held regularly, following the Scrum methodology, as shown in Table 1.

Table 1- Scrum meetings

Date	Meeting
12/12/2024	Sprint planning
17/12/2024	Daily meeting
19/12/2024	Daily meeting
26/12/2024	Daily meeting
02/01/2025	Daily meeting
07/01/2025	Daily meeting
09/01/2025	Daily meeting
10/01/2025	Daily meeting
11/01/2025	Daily meeting
12/01/2025	Sprint retrospective

1.3 Convention

To ensure consistency and organization during project development, clear conventions were established for naming, structuring and documenting the artifacts produced. The tasks managed in Jira were named in a standardized way, allowing easy identification of the type of activity and its relationship to the system requirements. In the code, the use of common practices in methods and variables was adopted. This choice ensured alignment with widely accepted standards in software engineering.

The structure of the code was planned to be modular, separating the interface, business logic and data persistence layers, which contributed to greater clarity and ease of development.

The project's source code was managed through a Git repository on GitHub. Only one repository was used, separating Frontend from Backend. The repository followed a clear organization of directories, and practices of frequent commits and detailed descriptions facilitated collaboration and organization within the team.

backend	Minor fixes in the models path
docs	feat: setup frontend + supabase integration + authenticatio...
frontend	feat: fetch individual product
.gitignore	Setup backend + supabase db integration example
README.md	feat: setup frontend + supabase integration + authenticatio...

Figure 1 - Project repository

2. Technology

The application was developed using the latest technologies to allow scalability, performance and easy maintenance.

The backend used the Express framework for Node.js, which facilitates the use of a flexible and efficient API, guaranteeing simple and fast endpoint management.

React was used on the frontend to provide an interactive and responsive interface. React allows components to be created in a reusable way, facilitating code optimization and development, as well as scalability. The application was developed with a responsive design, so that it is compatible with desktop and mobile use, providing the user with a consistent experience across different platforms.

For authentication and data management, Supabase was integrated, an open-source platform that offers a robust solution for the database, authentication and APIs. Supabase was chosen for its simple integration with the front and backend, allowing users to be created, permissions to be controlled and interaction with the database to be secure and scalable.

For requirements management and functionality planning, the Miro tool was used for story mapping, which made it easier to organize and prioritize tasks. The code was managed using Git, on GitHub, which enabled efficient and organized collaboration during development.

For managing subscriptions and one-time purchase payments, we use the Stripe API, which provides us with different payment methods in a secure and straightforward way.

3. Component Diagram

The solution consists of three main components previously defined in the SRS. These are the Web Portal (frontend), the API (backend) for feeding the Web Portal, and the Supabase service for using the Database and Authentication. The web portal can be used on both desktop and mobile devices.

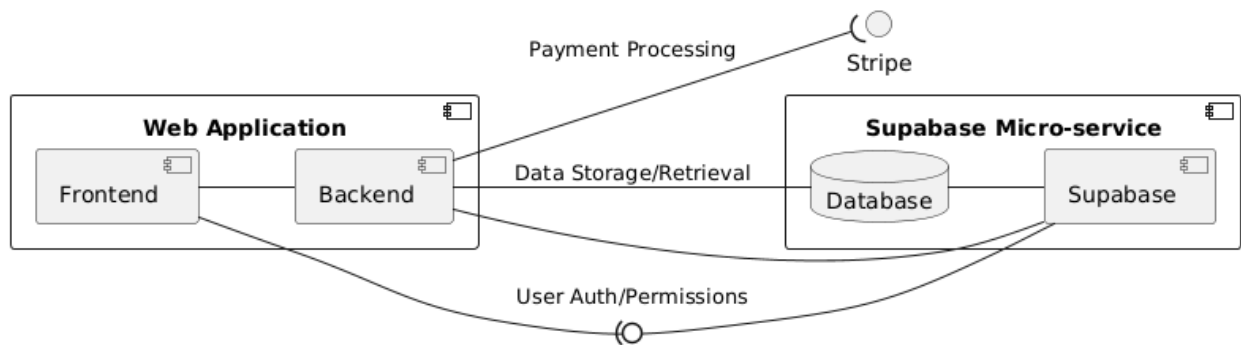


Figure 2 - Component diagram

4. Domain Model Diagram

Visual Paradigm Standard (copyrighted Instituto Superior de Engenharia do Porto)

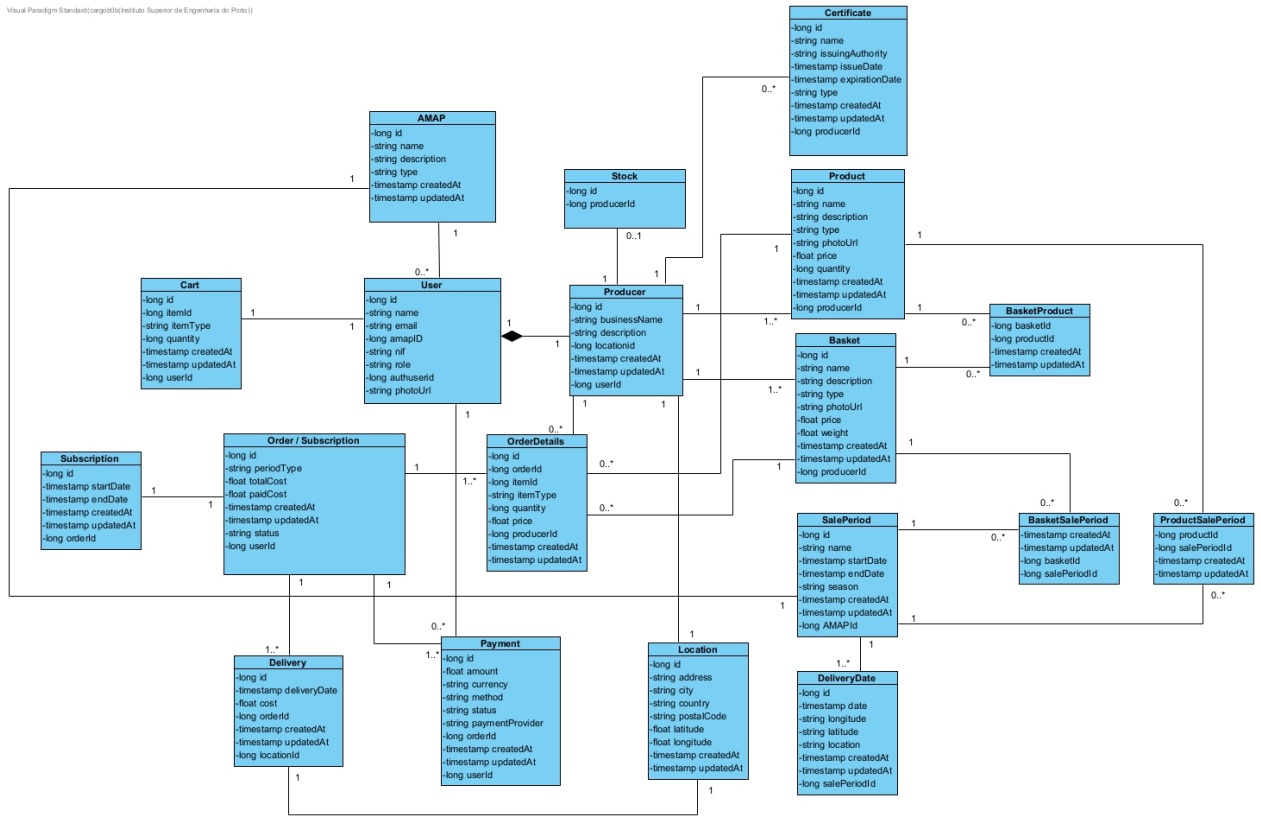


Figure 3 -Model diagram

The domain model represents the following business elements:

Core Business Elements:

- Users and Producers
- Users can belong to one AMAP organization
- Producers can create products and maintain stock
- Producers can have certificates (for organic or quality certifications)

Product Management:

- Products and Baskets
- Producers can create multiple products
- Products can be organized into baskets
- Products and baskets have sale periods
- Each product tracks details like price, quantity, and description

Order System:

- Orders/Subscriptions
- Users can create orders/subscriptions
- Orders contain order details linking to products
- Orders can have multiple payments
- Orders can be associated with deliveries

Location and Delivery:

- Delivery Management
- Orders can be linked to specific delivery dates
- Locations are tracked for deliveries
- The system maintains delivery dates and locations

Shopping Experience:

- Cart Management
- Users have shopping carts
- Carts can contain items before becoming orders

Payment Processing:

- Payment Handling
- Orders can have multiple payments
- Payments track status, currency, and payment methods

This is a system designed to:

- Connect producers directly with consumers
- Manage product inventory and distribution
- Handle subscriptions and regular deliveries
- Process payments and track deliveries
- Organize products into baskets for sale
- Maintain producer certifications and credentials

5. System Features

5.1 US01

This User Story outlines that a Producer should be able to define their product catalog, which includes specifying the name, a brief description, a photograph, a reference price, and the delivery units for each product. The product can either be **simple** (e.g., cured goat cheese) or **composite** (e.g., a basket of assorted vegetables). This functionality ensures that Producers can effectively manage and present their offerings to consumers.

To achieve this, Producers should have the ability to create and configure their product entries by providing all necessary details. The system should support uploading a photograph, setting a reference price, and defining delivery units, ensuring clarity and transparency for potential buyers. Additionally, the system should allow flexibility in categorizing products as either simple or composite, enabling Producers to represent their products accurately and comprehensively.

The person responsible for documenting and structuring this user story was Tomás Oliveira.

5.1.1 Story Mapping

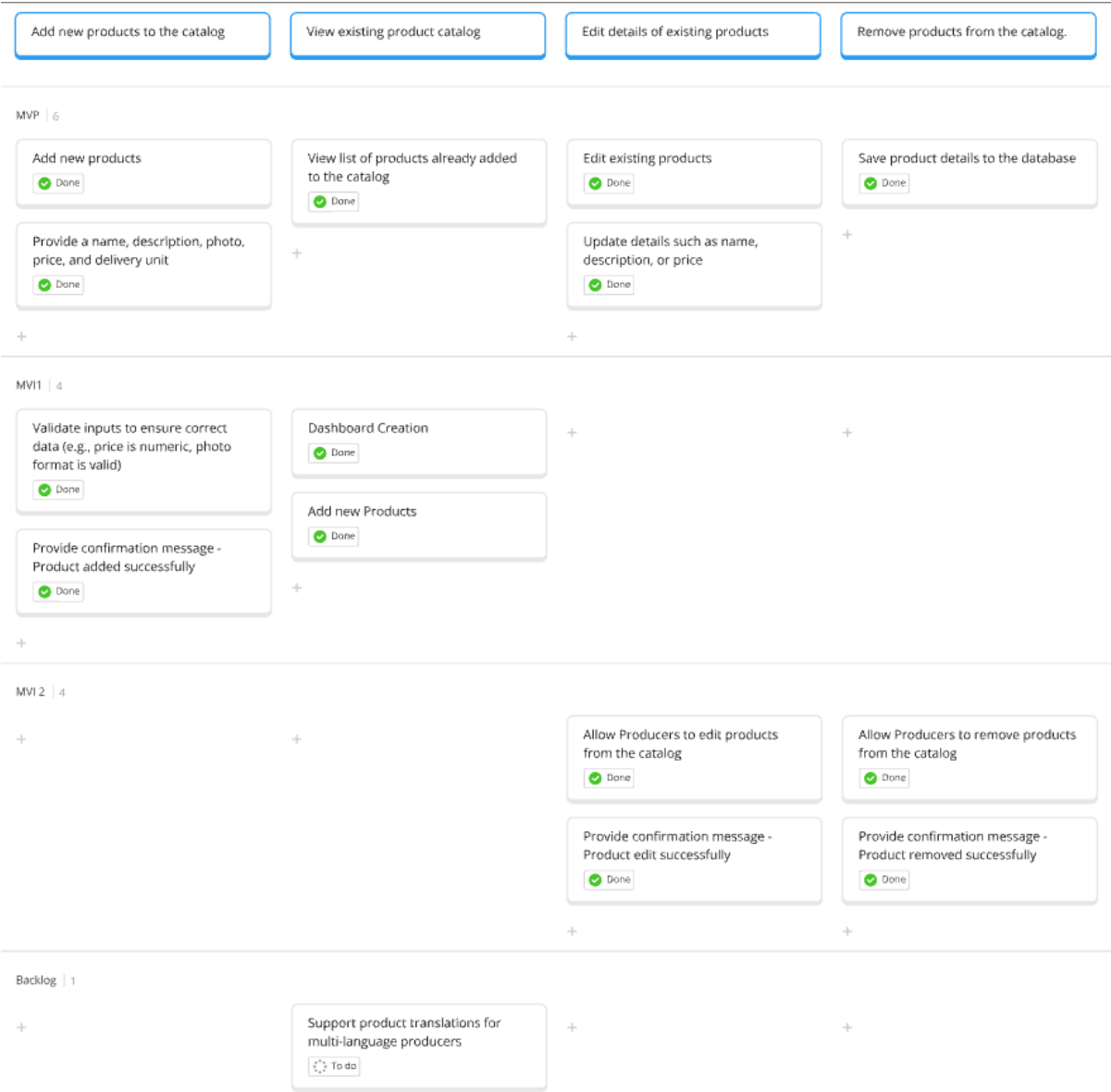


Figure 4 - Story mapping US1

This story mapping focuses on the management of a product catalog for producers. Key topics include **adding new products**, **viewing the existing product catalog**, **editing details of existing products**, and **removing products from the catalog**. These themes ensure producers can efficiently manage their product offerings with all necessary functionalities.

The **MVP** includes features such as adding new products to the catalog with essential details like name, description, photo, price, and delivery unit, viewing a list of products already added, and editing existing product details. It also ensures that changes are saved to the database.

For **MVI 1**, additional enhancements focus on data validation to ensure input accuracy (e.g., validating numeric prices and photo formats) and providing confirmation messages for successful product additions. This increment also includes the creation of a dashboard for improved navigation and organization.

In **MVI 2**, producers gain the ability to edit and remove products from the catalog, with confirmation messages for successful edits or removals. This increment emphasizes enhancing the flexibility and usability of the catalog management process.

Finally, the **Backlog** includes advanced features such as supporting product translations for multi-language producers, further expanding the system's accessibility and usability for a broader audience.

5.1.2 Development and Prioritization

In terms of user story planning, we started by prioritizing the implementation of product creation, where we can define the name, description, type, season with delivery dates, price, quantity, and photo.

Next, we focused on input validations and creating the dashboard that lists all products, including various search filters to enhance the user experience.

ERG-40	[FE] Create new product view	TO	DONE
ERG-41	[BE] Create new product endpoint	BS	DONE
ERG-22	[BE] Products endpoint	BS	DONE
ERG-20	[FE] Products list View	TO	DONE
ERG-48	[FE] Table sorting & filters	LS	DONE

Figure 5 - Jira User Stories

With medium priority, we defined product editing, product removal, and confirmation alerts for product creation and deletion (which ended up being implemented in Sprint 2).

In the backlog, we left the support for a multi-language platform.

5.1.3 Process View

On the homepage, when a Co-producer selects an AMAP, they are redirected to the AMAP catalog page, displaying all the registered products



Figure 6 - Dashboard view

When the user enters the page, he can apply various search filters to find the product or click the button in the top-right corner if he wants to register a product.

The screenshot shows the AMAP catalog view with a dark blue sidebar on the left containing navigation links: AMAP's, Cart, History, Profile, Amap Management, and Sign Out. The main content area has a dark background and features a table of products. Above the table, there is a search bar labeled 'Search product' and a 'Create Product' button. The table has four columns: Product Name, Product Type, Season, and Product Price per kg. The table contains 20 rows of product data.

Product Name	Product Type	Season	Product Price per kg
SummerTest	type1	Season test edited	123 €
TesteProduct	type1	Season test edited	123 €
WinterTest	type1	Season winter	123 €
Teste	type1	Season test edited	123 €
Laranja	type1	Season days winter test	10 €
Lichia	type1	Season winter	4.99 €
Batata	type1	Sigma Season	5 €
US03	type1	Teste13456	10 €
Laranja	type1	N/A	10 €
Apple	type1	N/A	15 €
Cereal	type1	N/A	12.5 €
Tomato	type2	N/A	20 €
Banana	type1	N/A	10 €
Mushrooms	type1	N/A	12.5 €
Milk	type1	N/A	12.5 €
Pizza de Pepperoni	type1	N/A	10 €
Chicken Breast	type2	N/A	25 €
TesteBjeras	type1	N/A	123123 €
Kiwi	type1	N/A	10 €
Pizza Margarita	type1	N/A	9 €

Figure 7 - Amap catalog view

AMAP's

Cart

History

Profile

Amap Management

Sign Out

Register Product

Name

Description

Type

Select	Season Name	Start Date	End Date
☐	Season days winter test	05/11/2024	05/02/2025
☐	Create Season Test	05/11/2024	05/02/2025
☐	Test Season	11/01/2025	23/01/2025
☐	Sigma Season	11/01/2025	14/05/2025
☐	123	11/01/2025	31/01/2025
☐	SeasonTest123	25/01/2025	28/03/2025
☐	Teste13456	16/01/2025	14/02/2025
☐	Season winter	05/11/2024	05/02/2025
☐	Season test edited	05/11/2024	05/02/2025

Price

Quantity

Photo

No photo selected

Select Photo

Register Product

Figure 8 - Register product view

After the user clicks on Create Product, he is redirected to a product registration form that allows the user to input different details about a product. It includes fields for the product's name, description, type, price, and quantity. There is also a table displaying multiple seasons, each with a season name, start date, and end date, where the user can select one season using radio buttons. Additionally, there is a section to upload or select a photo for the product, with a button labeled Select Photo. Once all the necessary information is filled in, the user can click the Register Product button to finalize and save the product details. The layout is designed to be user-friendly and intuitive.

5.2 US02

This User Story outlines that an AMAP Manager should be able to define a Subscription Period with a specific duration (e.g., monthly). The Subscription Period must include start and end dates as well as predefined delivery dates that occur within the specified period. This ensures that all deliveries are planned and organized efficiently. To achieve this, the AMAP Manager should have the ability to create and configure the Subscription Period by selecting the duration, setting start and end dates, and defining delivery dates. These configurations should ensure compatibility with the producers' schedules.

The person responsible for documenting and structuring this user story was Tomás Oliveira.

5.2.1 Story Mapping

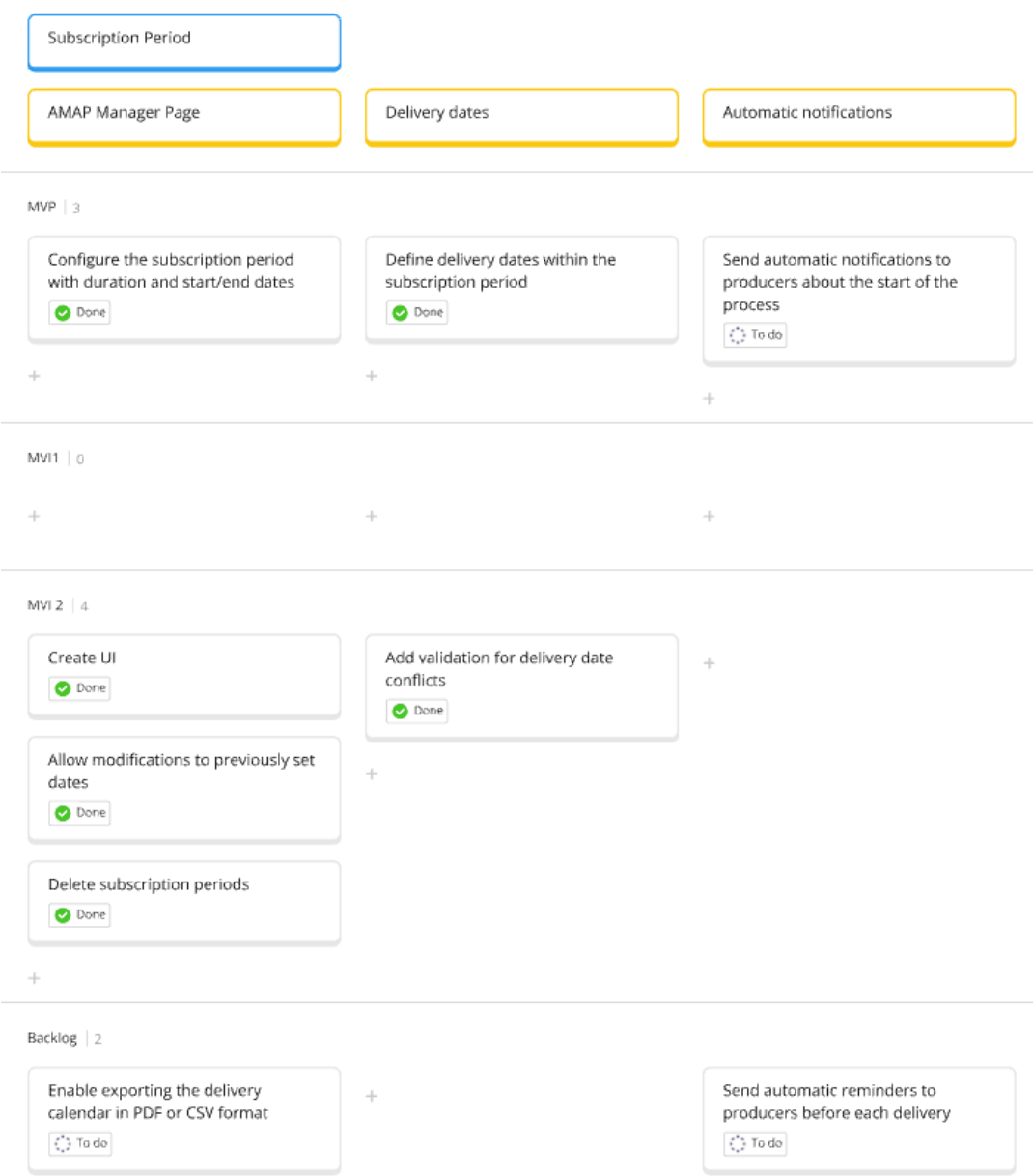


Figure 9 - Story mapping US 2

This story mapping focuses on key themes such as the Subscription Period, which involves defining the start and end dates of the subscription, the AMAP Manager Page as a central interface for managing subscription periods, Delivery Dates for organizing schedules within the subscription period, and Automatic Notifications to inform producers about critical events.

The MVP includes configuring the subscription period with duration and start/end dates, defining delivery dates within the subscription period, and sending automatic notifications at the start of the process. No tasks are outlined for MVI 1, while MVI 2 focuses on building a UI for subscription and delivery management, adding validation to prevent delivery date conflicts, enabling modifications to previously set dates, and allowing subscription periods to be deleted.

Finally, the Backlog includes advanced features such as exporting the delivery calendar in PDF or CSV format and sending automated reminders to producers before each delivery.

5.2.2 Development and Prioritization

In terms of planning the User Stories, the initial priority was to create the AMAP Management page. We also developed the backend endpoint that allowed us to store the values in the respective tables. From a UI perspective, we created a route for a page dedicated to creating a Season, where it was necessary to specify the name, start date, end date, and the corresponding season of the year for the Season being created.

 ERG-129 [FE] AMAP Management Page	   TO DONE 
 ERG-86 [BE] Seasons CRUD	   B5 DONE 
 ERG-130 [FE] Create Seasons	   TO DONE 
 ERG-90 [BE] Subscription CRUD	   B5 DONE 
 ERG-89 [FE] Subscription Period	   TO DONE 

Figure 10 - Jira User Stories

After addressing these critical points, we prioritized date validations to ensure there were no conflicts between the start and end dates. Additionally, we enabled the editing and deletion of created Seasons, specifically for AMAP Managers.

 ERG-128 [FE] Add validation for delivery date conflicts	   TO DONE 
 ERG-131 [FE] Edit and Remove Season	   TO DONE 

Figure 11 - Jira User Stories

Finally, as lower-priority tasks, we left sending automatic reminders to producers before each delivery and enabling the export of the delivery calendar in PDF or CSV format in the backlog.

ERG-99	[FE] Send automatic reminder to producers before each delivery	TO DO
ERG-132	[FE] Export of the delivery calendar in PDF or CSV format	TO DO

Figure 12 - Jira User Stories

5.2.3 Process View

On the homepage, when the AMAP Managers clicks on the AMAP Management tab in the Sidebar, they are redirected to a page where they can view relevant information for their role and can define subscription periods



Figure 13 - Dashboard view

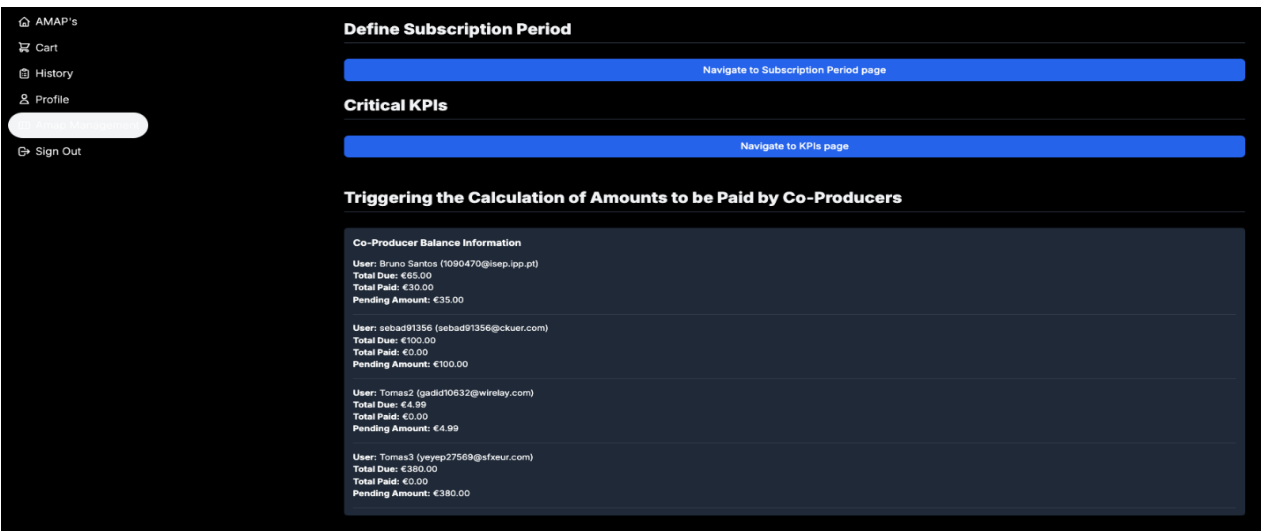


Figure 14 - Amap management view

When the AMAP Manager navigates, they are redirected to a page where it is possible to create seasons with four inputs: the season name, the start date, and the end date, which are controlled by a date-picker with input validations allowing only valid days to be selected. The user must also select the season of the year to which the season will belong before creating it.

On this page, the AMAP Managers can also view a list of already created seasons, where they can edit and remove them.

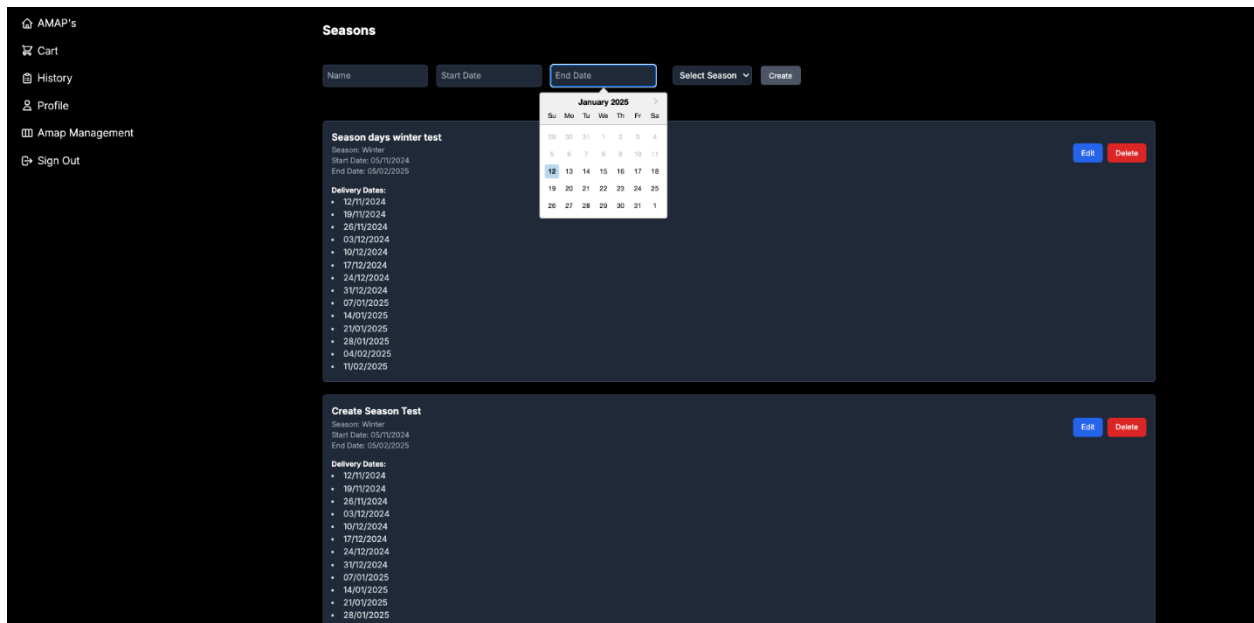


Figure 15 - Season configs view

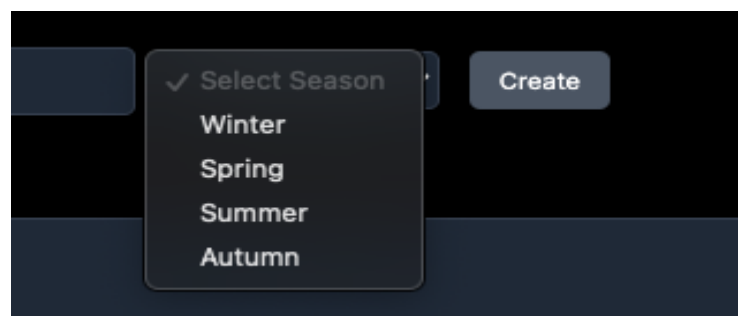


Figure 16 - Season selection view

5.3 US03

This User Story outlines that a Producer should be able to associate a **Subscription Period** with their own Product or Basket. The Subscription Period must be selected from a list of predefined options created by the AMAP to which the Producer belongs.

To achieve this, the Producer should be able to define delivery dates during the creation or update of an item by selecting the Subscription Periods that align best with their production schedule. This ensures that the Product or Basket can be delivered on the specified dates.

Additionally, the Producer should have the ability to select their preferred payment methods. This allows them to receive payments in their chosen method whenever a Coproducer purchases one of their Products or Baskets.

The person responsible for documenting and structuring this user story was Hellen Santos.

5.3.1 Story Mapping

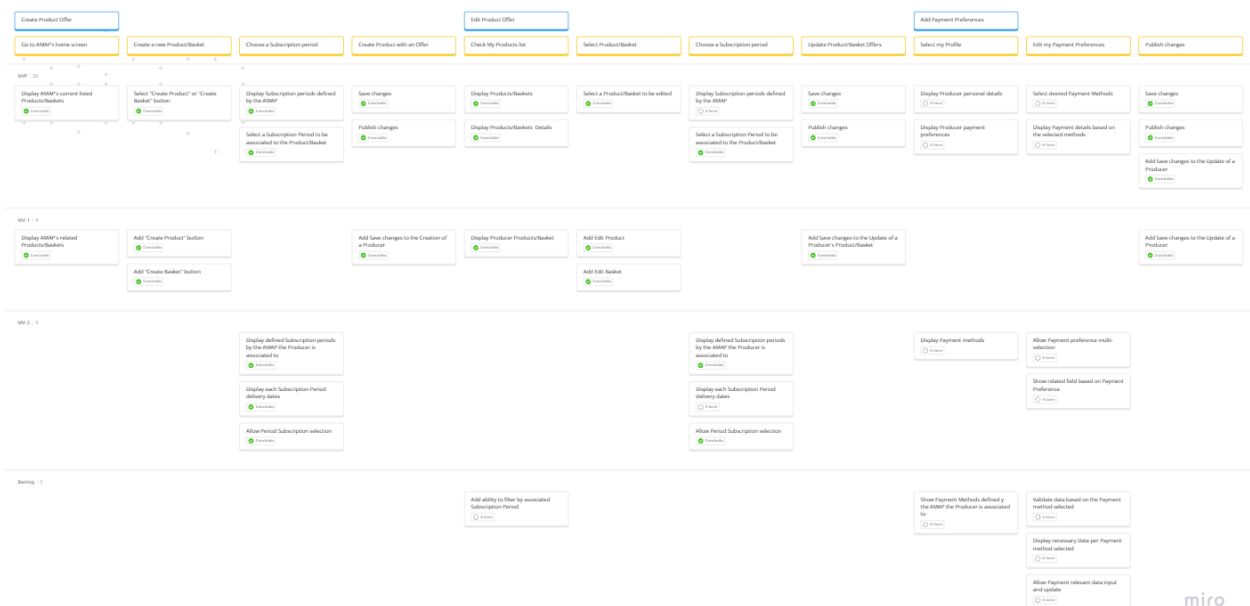


Figure 17 - Story mapping US3

The story map for this User Story outlines the main user actions related to Producers managing their Product Offers, Subscription Periods, and Payment Preferences in our application. The goal is to provide Producers with a seamless process for creating and managing Products or Baskets, associating them with Subscription Periods, and defining payment preferences.

The Minimum Viable Product part of this Story Mapping focuses on enabling Producers to create and manage their Products and/or Baskets by associating them with predefined Subscription Periods established by the AMAP they are part of.

All the tasks exposed in the Story Mapping are marked as “Done” or “To-Do” based on if they were finished in the Sprint they were planned to - MVI1 for Sprint 1 and MVI2 for Sprint 2.

With those details, this Story Mapping ensures the system delivers the core functionalities which are essential for Producers to offer their items to Coproducers effectively. It also provides a structured view of the Producer’s journey in managing their offerings by emphasizing the incremental feature delivery through MVP, MV1, MV2, and an additional backlog of items for future improvements.

5.3.2 Development and Prioritization

The primary focus during Sprint 1 was to establish a robust foundational structure for the application, ensuring that by the end of the Sprint, the system would be prepared for the upcoming implementations. For this User Story, the key objectives were to enable the creation of Products and Baskets, as well as provide Producers with the ability to view the Products and Baskets they have created for the AMAP they belong to.

These foundational features were crucial for setting up the system’s core functionality and facilitating future development across different requirements.

 ERG-26	[FE] Product details view	 - 	TO	CONCLUÍDO ▾
 ERG-28	[BE] Product details endpoint	 - 	BS	CONCLUÍDO ▾
 ERG-50	[DB] Product Details Seeder	 - 		CONCLUÍDO ▾
 ERG-20	[FE] Products list View	 - 	TO	CONCLUÍDO ▾
 ERG-21	[FE] Basket list View	 - 	TO	CONCLUÍDO ▾
 ERG-22	[BE] Products endpoint	 - 	BS	CONCLUÍDO ▾
 ERG-23	[BE] Basket endpoint	 - 	BS	CONCLUÍDO ▾

Figure 18 - Jira User Stories

With the foundational features for Product and Basket creation and visualization completed during Sprint 1, Sprint 2 shifted its focus to establishing the crucial association between the Subscription Periods defined by the AMAP (as outlined in User Story 02) and the Products and Baskets created by the Producer.

The priority tasks for this Sprint were centered on enabling Producers to edit their Products and Baskets, as well as modifying the creation process for Products and Baskets to allow for the seamless association of items with Subscription Periods. This step was key to ensuring the system's functionality aligned with the AMAP's subscription model and the Producers' offerings, as it was defined for this User Story.

	ERG-116	[FE] Edit Product/Basket Delivery dates				CONCLUÍDO ▾
	ERG-117	[BE] Edit Product/Basket Delivery dates				CONCLUÍDO ▾
	ERG-126	[BE] Fetch Products and Baskets Sale periods				CONCLUÍDO ▾
	ERG-127	[FE] Display Products and Baskets Sale peri...				CONCLUÍDO ▾
	ERG-87	[FE] Season - Delivery dates				CONCLUÍDO ▾
	ERG-118	[FE] Associate Products/Baskets to Sales Per...				CONCLUÍDO ▾
	ERG-119	[BE] Delivery Dates endpoint			BS 	CONCLUÍDO ▾
	ERG-62	Edit producer products inside his profile page				CONCLUÍDO ▾ 

Figure 19 - Jira User Stories

With the association between Subscription Periods and Products/Baskets now established, the Producer is fully capable of creating and editing Product/Basket offers. However, this User Story also outlined that the Producer should have the ability to define their own payment preferences, including the option for fractional payments.

At the current state of the system and based on the discussions with the AMAP's representative, it is known that AMAP Porto currently cannot accept payments and distribute them to the respective Producers, so these features are essential for providing the Producer with greater flexibility and control over the type of transactions they accept from the Co-producers.

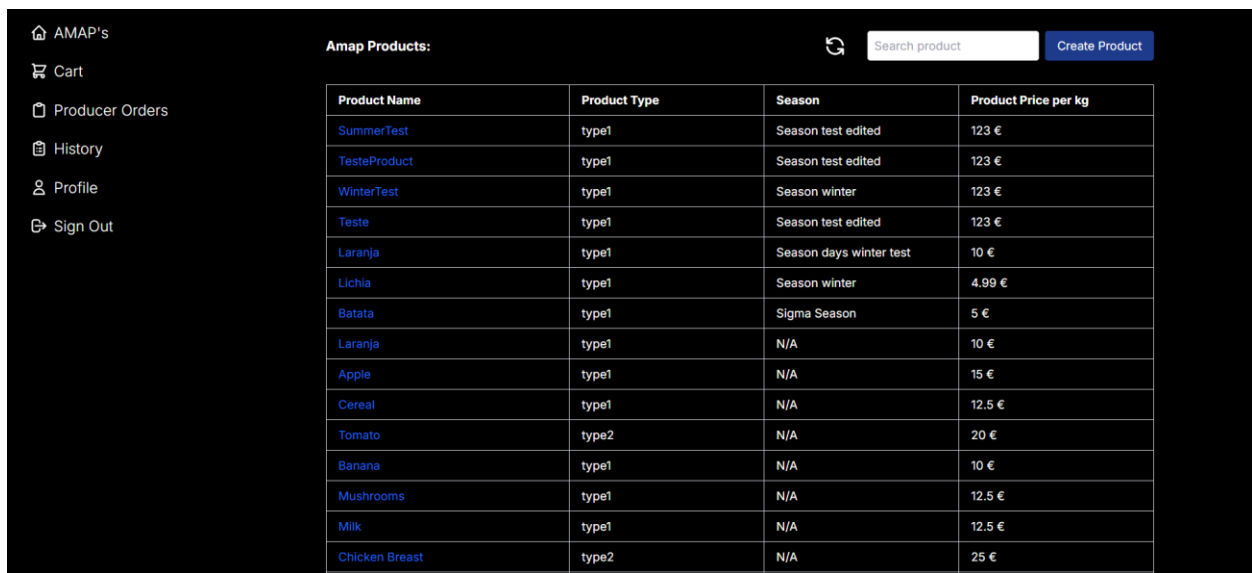
	ERG-111	[BE] & [FE] Support fractional payments				TAREFAS PENDENTES ▾
	ERG-112	[BE] & [FE] Support multiple payment methods				TAREFAS PENDENTES ▾

Figure 20 - Jira User Stories

While the system uses Stripe for online payment processing, and Stripe supports conditional payment processing, it does not natively handle fractional payments. Which means that implementing fractional payments would require a custom solution within the application.

Additionally, enabling fractional payments and offering payment method preferences would necessitate significant changes to previously developed requirements. These tasks were deemed low priority for this User Story, and during our planning, it was determined that completing them within the Sprint timeline and our personal constraints would not be possible. As a result, the tasks related to this aspect of the User Story were moved to the backlog as future improvements.

5.3.4 Process View



Product Name	Product Type	Season	Product Price per kg
SummerTest	type1	Season test edited	123 €
TesteProduct	type1	Season test edited	123 €
WinterTest	type1	Season winter	123 €
Teste	type1	Season test edited	123 €
Laranja	type1	Season days winter test	10 €
Lichia	type1	Season winter	4.99 €
Batata	type1	Sigma Season	5 €
Laranja	type1	N/A	10 €
Apple	type1	N/A	15 €
Cereal	type1	N/A	12.5 €
Tomato	type2	N/A	20 €
Banana	type1	N/A	10 €
Mushrooms	type1	N/A	12.5 €
Milk	type1	N/A	12.5 €
Chicken Breast	type2	N/A	25 €

Figure 21 - Amap catalog view

For the creating a Product with a Subscription Period associated to it, first you need to navigate to the AMAP's main page as a Producer, and then select the "Create Product" or "Create Basket" buttons.

AMAP's

Cart

Producer Orders

History

Profile

Sign Out

Register Product

Name

US03

Description

Batatas Organicas

Type

Vegetal

Select	Season Name	Start Date	End Date
☐	Season days winter test	05/11/2024	05/02/2025
☐	Create Season Test	05/11/2024	05/02/2025
☐	Test Season	11/01/2025	23/01/2025
☐	Sigma Season	11/01/2025	14/05/2025
☐	123	11/01/2025	31/01/2025
☒	SeasonTest123	25/01/2025	28/03/2025
☐	Teste13456	16/01/2025	14/02/2025
☐	Season winter	05/11/2024	05/02/2025
☐	Season test edited	05/11/2024	05/02/2025

Selected Season's Delivery Dates

01/02/2025 | 08/02/2025 | 15/02/2025 | 22/02/2025 | 01/03/2025 | 08/03/2025 | 15/03/2025 | 22/03/2025 | 29/03/2025

Price

10

Quantity

5

Photo

No photo selected

Select Photo

Register Product

Figure 22 - Register product

The buttons will redirect to a Register page, which will allow the Producer to input all the relevant Product/Basket information, as well as select one of the Subscription periods defined by the AMAP the Producer is associated with.

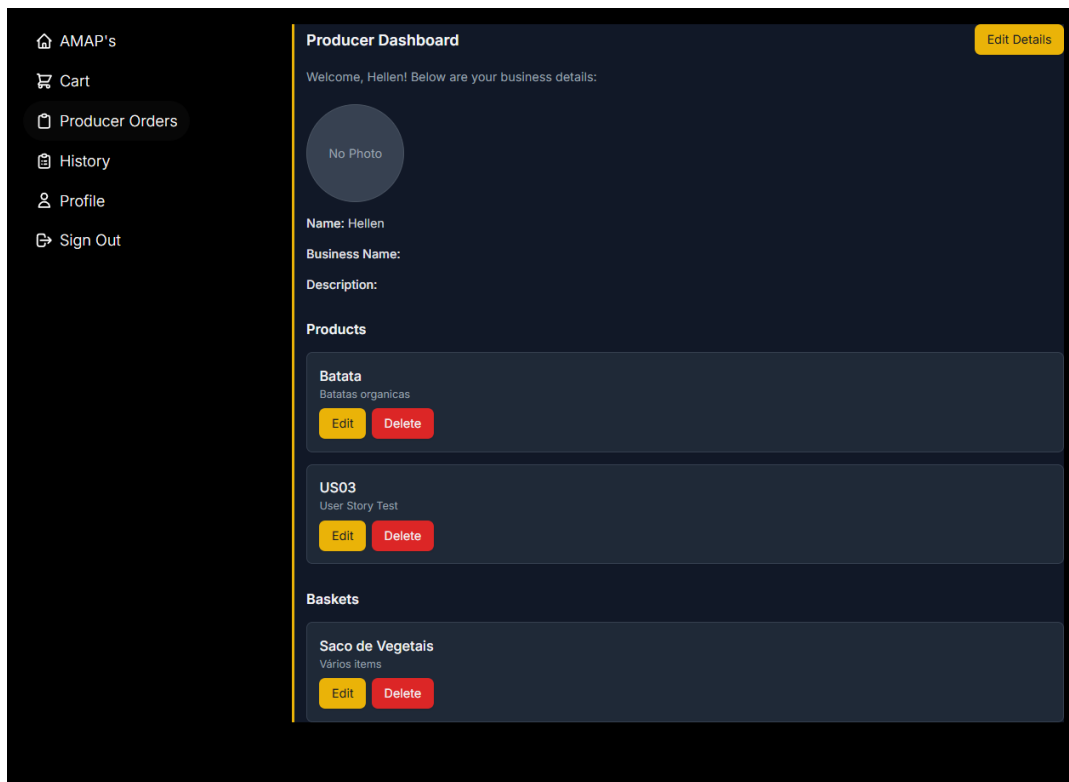


Figure 23 - Producer profile view

To edit or check the Subscription period the Products and Baskets are associated with, the producer should go to its own profile and edit any of the items he has created.

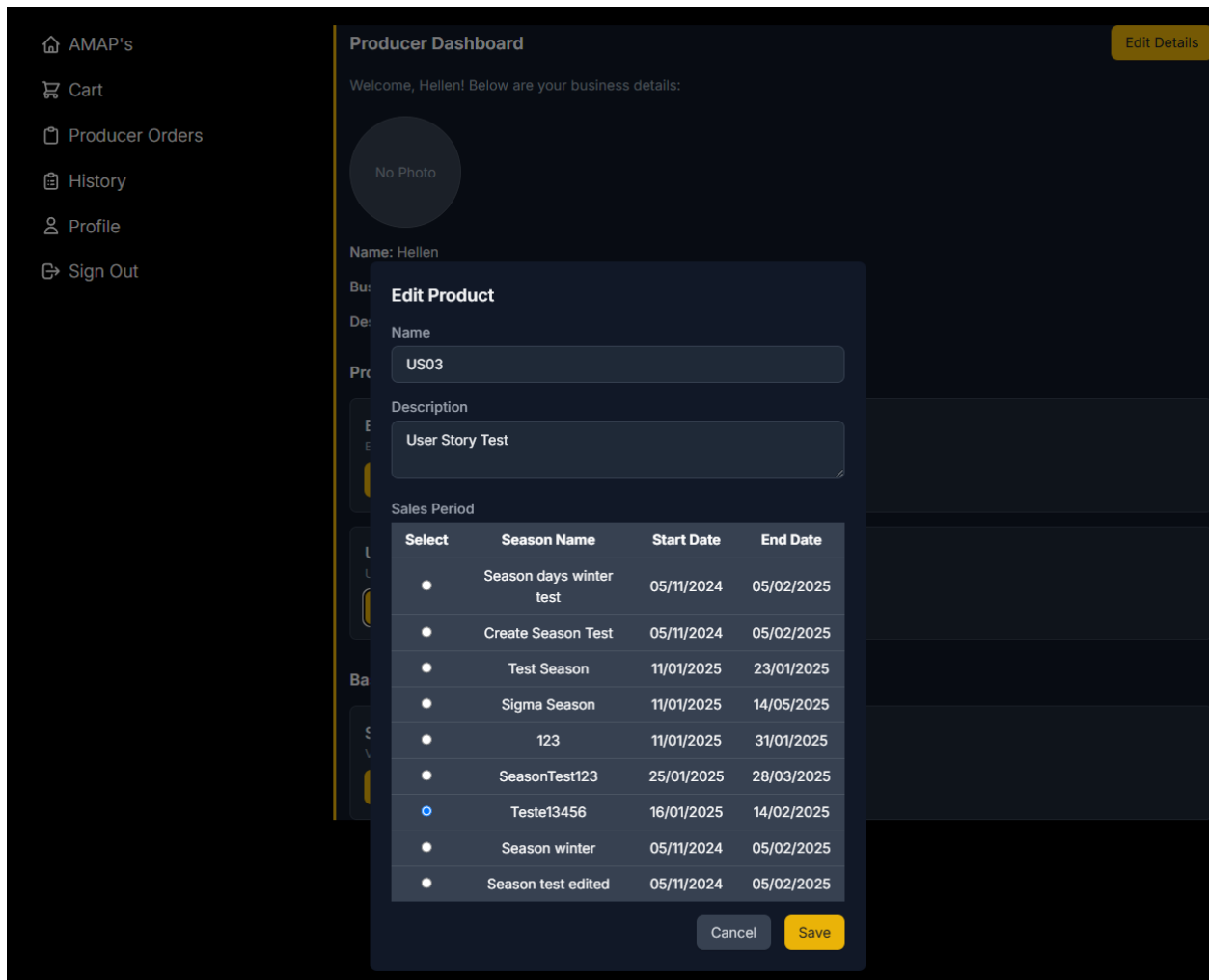


Figure 24 - Product edit view

The “Edit Item” modal will show the associated Subscription period, and it will allow the Producer to change it as needed, by changing the chosen Subscription period and then clicking “Save”.

5.4 US04

In this user story, the user with the Coproducer role is intended to be able to subscribe to products and/or baskets made available by the producers. They select the quantity they want to receive on each date and choose the payment method depending on the options available.

To achieve this functionality, the user must be able to view the list of products and/or baskets made available by producers. On the product/basket detail sheet, an option to subscribe and another to add to the cart should be provided.

If the user chooses to subscribe, they must select the period and quantity they want.

If the user choose to add to cart, you will be asked for the quantity want. At any time, the user must be able to check the items already added to the cart. User must validate the items, change the quantity if necessary or even remove items if don't want any of the ones already added.

When checking out or subscribing, the user is directed to the payment service to make the payment.

The person responsible for documenting and structuring this user story was Bruno Santos.

5.4.1 Story Mapping

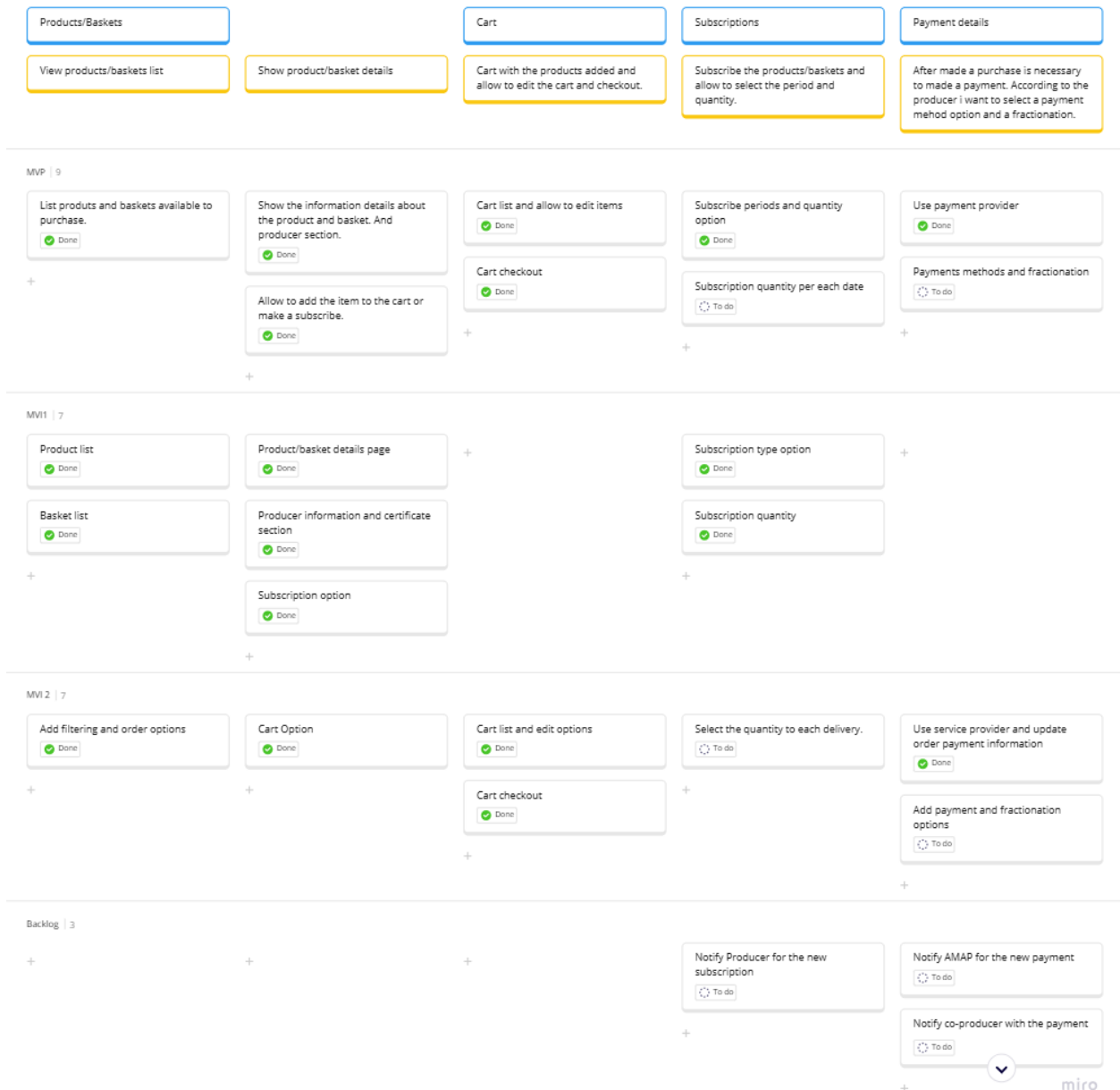


Figure 25 - Story mapping US04

This story mapping focuses on the subscription and purchase of products by the Co-Producer, involving the whole process of subscribing to the products available and then paying for them.

It is divided into the product listing, the purchase, the subscription and finally the payment.

The MVP includes the list of products/baskets, their details showing the relevant information, as well as their producer. This allows you to make your purchase or subscription. When you buy, you can manage the quantity you want and your cart. In the case of a subscription, you can select the subscription period and the quantity you want. In the final step, you can make the payment according to the purchase made.

In MVI 1, the focus was on the basic interface so that it would be possible to purchase/subscribe to products with their list and detailed information. It already includes the subscription.

In MVI 2, the listing of products and baskets has been improved by adding filters. Also in the addition of a Cart, making it possible to add to the Cart, as well as the creation of an interface to manage the Cart and enable its checkout. At the payment level, integration with a payment service provider is needed, as well as an interface.

Finally, the backlog includes notifications to producers of subscriptions made, as well as payment status.

5.4.2 Development and Prioritization

In terms of prioritization, the necessary basis would be the list of products and baskets made available by AMAP producers.

The list of available products and baskets, as well as their detail page and producer information section, was implemented in Sprint1. The possibility of subscribing was also implemented in Sprint1.

Priority 1: Ensure that the product and basket listings are working, improving them by applying filters and sorting.

Priority 2: Implement a Cart.

Priority 3: Implement a payment service and manage payments.

Once the listing and subscription were implemented, the priority would be the possibility of the Cart. It would be necessary to implement a view for the Cart and the corresponding option in the menu, so that it would be possible to manage the items in the Cart. It was also necessary to add the add to cart option to products and baskets. Finally, in the cart, the possibility of checkout to make the respective purchase. All these options would also require the implementation of endpoints to handle the actions provided.

	ERG-107	[FE] Add items to cart				DONE ✓
	ERG-108	[BE] Cart CRUD				DONE ✓
	ERG-109	[FE] Cart checkout				DONE ✓

Figure 26 - Jira User Stories

After implementing Cart, the next priority would be Payment. This is necessary and important in the application.

For payment, it would be necessary to implement a service provider to handle with the payments. It would also be necessary to provide payment options and split payments according to the options provided.

	ERG-110	[BE] Add payments				DONE ✓
	ERG-111	[BE] & [FE] Support fractional payme...				TO DO ▼
	ERG-112	[BE] & [FE] Support multiple paymen...				TO DO ▼

Figure 27 - Jira User Stories

5.4.3 Process View

The graphical interfaces produced for this User Story are as follows:

The user starts by selecting the desired AMAP.



Figure 28 - Dashboard view

Once the AMAP has been selected, the user will see a list of all the products and baskets offered by the respective AMAP. You can use filters and sorting to find the items you want.

Amap Products:

Search product

Create Product

Product Name	Product Type	Season	Product Price per kg
SummerTest	type1	Season test edited	123 €
TesteProduct	type1	Season test edited	123 €
WinterTest	type1	Season winter	123 €
Teste	type1	Season test edited	123 €
Laranja	type1	Season days winter test	10 €
Lichia	type1	Season winter	4.99 €
Batata	type1	Sigma Season	5 €
Laranja	type1	N/A	10 €
Apple	type1	N/A	15 €
Cereal	type1	N/A	12.5 €
Tomato	type2	N/A	20 €
Banana	type1	N/A	10 €
Mushrooms	type1	N/A	12.5 €

Figure 29 - Amap catalog view

Clicked in a list item, goes to the details view of the product, with a description, price, quantity available and product image. There is also a producer section, with a description and if they have their organic certificate. Have the option to add the item to the Cart and subscribe the item using the buttons provided.

Product Details:

Apple

Description for Apple

Price: 15 €

Quantity Available to Produce: 30



Add product to cart

Subscribe to product

Producer Details:

Business Name:

Contact Email: tom.hardy@example.com

Certificate: Rei dos Frangos

Issuing Authority: Luis Filipe Vieira

Issue Date: 13/11/2024

Expiration Date: 15/04/2025

Figure 30 - Product details view

In the case of the subscription option, a view is shown with the subscription options, and you need to select the subscription period and the desired amount.

A screenshot of a web form titled "Subscription" in white text on a black background. The form itself has a dark blue background. It contains the following elements: "Product Name: Eggs" in white text; "Subscription Type:" followed by a white dropdown menu showing "Weekly"; "Quantity:" followed by a white text input field containing the number "1"; and a green button at the bottom with the text "Subscribe Basket" in white.

Figure 31 - Subscripion view

In the case of the Cart option, a view will be shown for selecting the quantity.

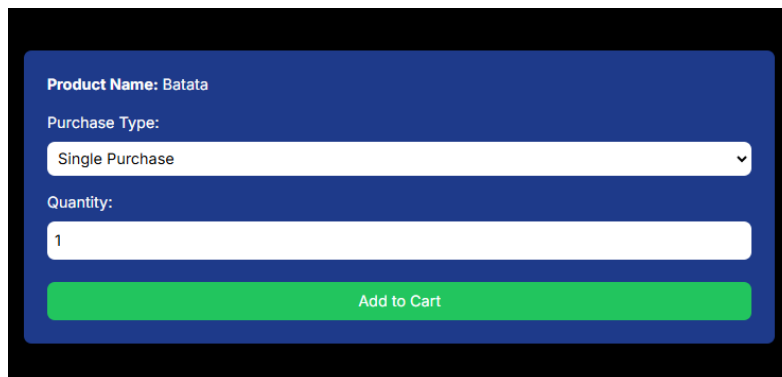
A screenshot of a web form titled "Add to Cart" in white text on a black background. The form has a dark blue background. It contains the following elements: "Product Name: Batata" in white text; "Purchase Type:" followed by a white dropdown menu showing "Single Purchase"; "Quantity:" followed by a white text input field containing the number "1"; and a green button at the bottom with the text "Add to Cart" in white.

Figure 32 - Add to cart view

When you are adding items to the Cart, you can select the option at any time by accessing the side menu, showing the Cart view,

In this view you can manage the items you have added, such as changing the quantity you want or removing an item. If everything is OK and the user wants to check out, the option to finalize the purchase is available.

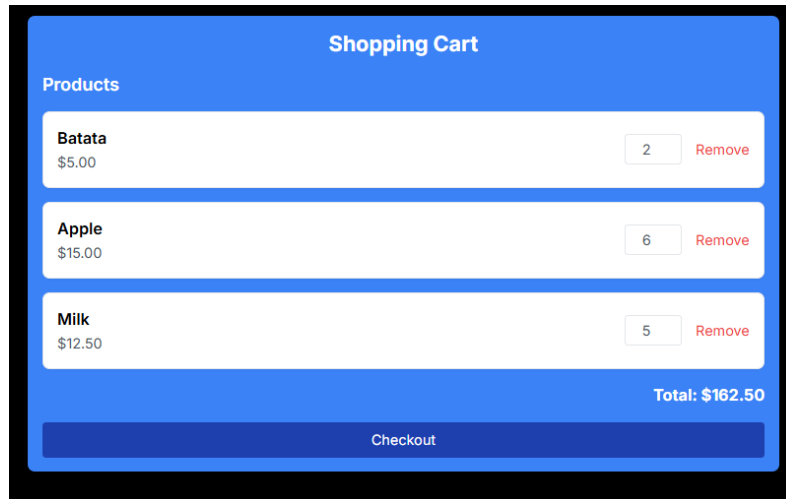


Figure 33 - Cart view

Allow change the quantity and remove the item from the cart.

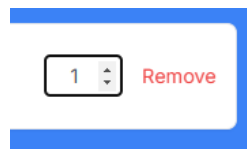


Figure 34 - Quantity option view

After subscribing or checking out of Cart, the user is directed to the payment view, the next step necessary to complete their purchase / subscription.

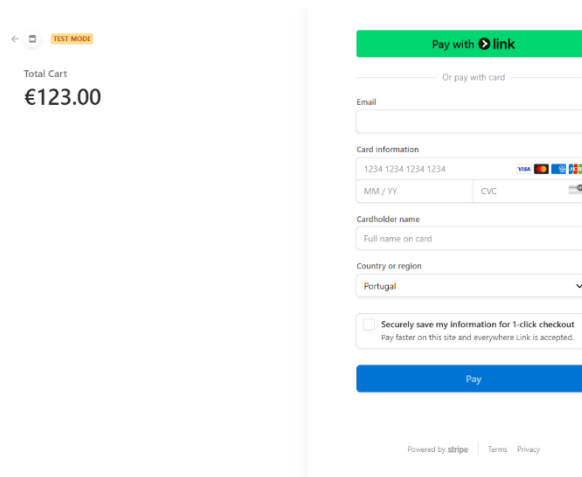


Figure 35 - Payment view

5.5 US05

In this user story, a user with the role “AMAP Admin” must be able to trigger the amount to be paid by the co-producers and the amount to be received by the producers, taking into account the subscriptions and payments made.

To implement this, it is necessary for a user with the AMAP Admin role to have the option available in the menu. It is necessary to have a view that shows all outstanding amounts by Co-producer and Producer. The amounts to be calculated must take into account the amounts subscribed and the amounts paid.

The person responsible for documenting and structuring this user story was Bruno Santos.

5.5.1 Story Mapping

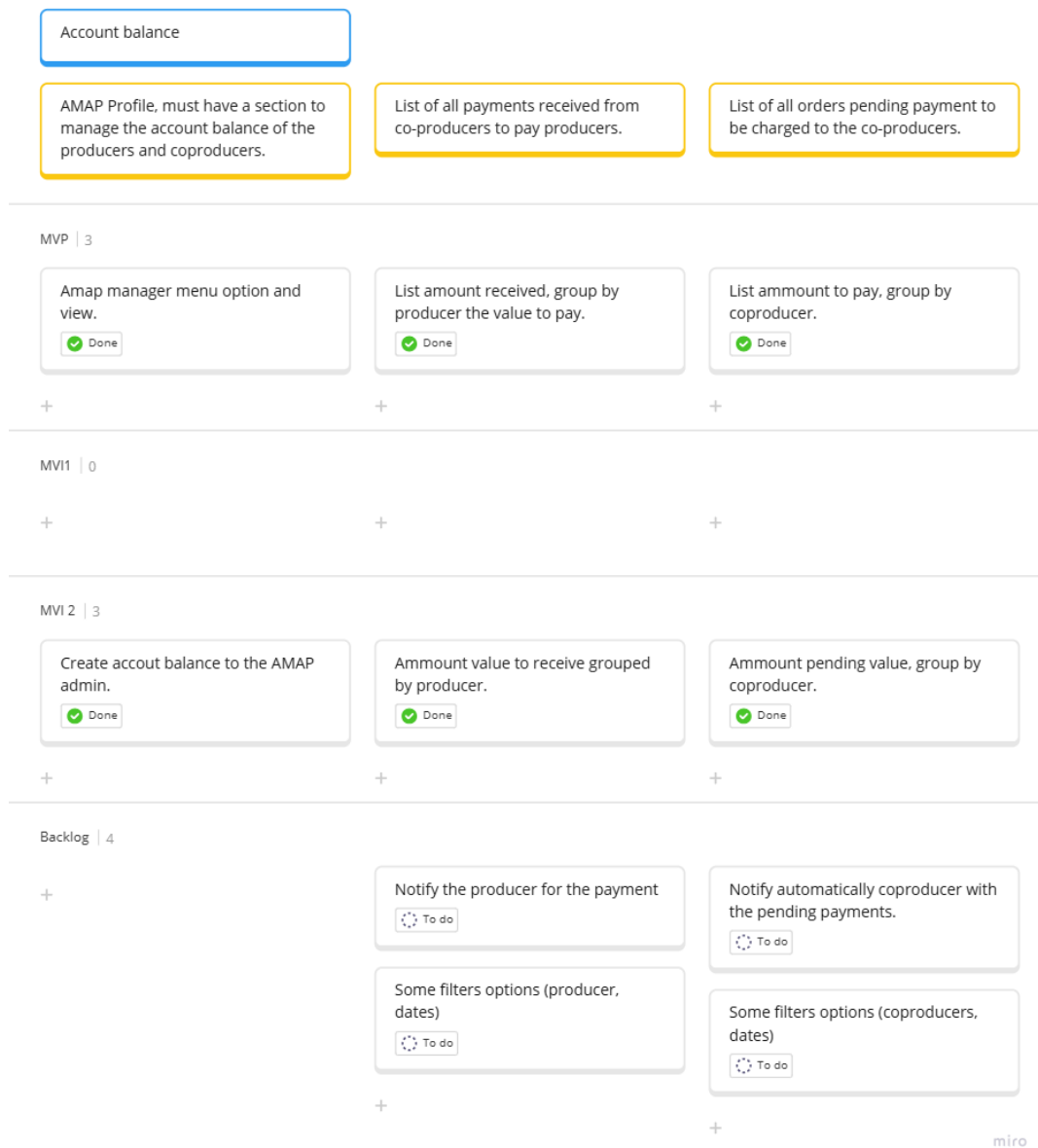


Figure 36 -Story mapping US05

This story mapping focuses on the view for an AMAP Admin role to obtain the amounts to be received by the co-producers and paid to the producers.

The MVP includes the option in the menu and view for the AMAP admin role, so that the outstanding amounts can be made available. The list of amounts is grouped by coproducer and by producer.

There were no tasks in MVI 1, while in MVI 2 we have the task of creating the AMAP Management view, as well as a list of the amounts to be paid by coproducers and another list of the amounts to be received by producers.

Finally, the backlog includes notifications to the co-producers of the amounts to be paid and to the producers of the amounts to be received. And to make things even better, we've also added date filters.

5.5.2 Development and Prioritization

In terms of development and prioritization, this user story has been shared with another shared user story from AMAP Management, with the priority being the option in the menu for access and the respective view.

Priority 1: AMAP Management menu option and view.

Priority 2: Implement the co-producers' current account.

Priority 3: Implement the Producers' current account.

The main priority would be the option in the menu and the corresponding AMAP Management view.

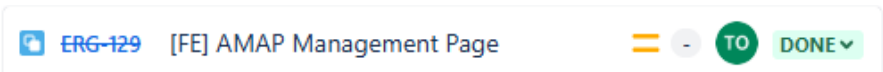


Figure 37 - Jira User Stories

We would then have the two options, the Co-producer's and the Producer's balance account, with the same level of priority, but with no payments made, there are no payments to be received. Priority was given to the list of amounts payable by the Co-Producers.

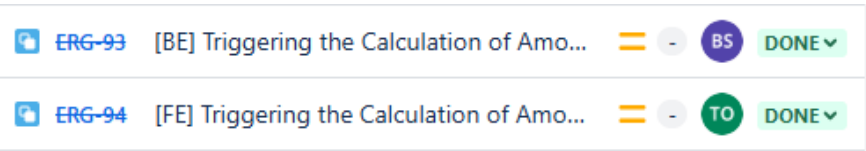


Figure 38 - Jira User Stories

Finally, we have a list of the amounts to be received by the Producers.

ERG-95	[BE] Triggering the Calculation of Amo...	BS	DONE
ERG-96	[FE] Triggering the Calculation of Amo...	TO	DONE

Figure 39 - Jira User Stories

5.5.3 Process View

The graphical interfaces produced for this User Story are as follows:

To access to this option, the user must have a AMAP Admin role, this way have the option “AMAP Management” in the menu.

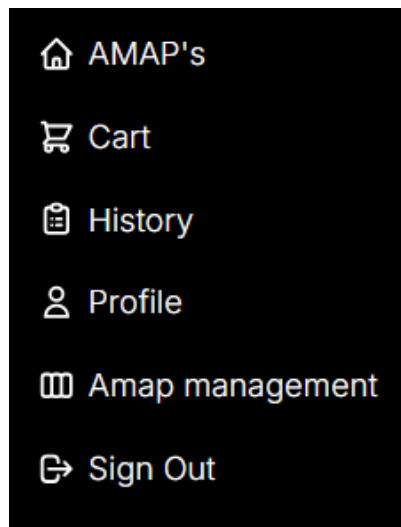


Figure 40 - Menu view

The AMAP Management view, have a section with the Amounts to be Paid by Co-producers, with the information by co-producer, the total value of the subscriptions, the value already paid, and the pending value necessary to pay.

Amounts to be Paid by Co-Producers
Co-Producer Balance Information User: Bruno Santos (1090470@lsep.ipp.pt) Total Due: €65.00 Total Paid: €30.00 Pending Amount: €35.00
User: sebad91356 (sebad91356@ckuer.com) Total Due: €100.00 Total Paid: €0.00 Pending Amount: €100.00
User: Tomas2 (gadid10632@wirelay.com) Total Due: €4.99 Total Paid: €0.00 Pending Amount: €4.99
User: Tomas3 (yeyep27569@sfxeur.com) Total Due: €380.00 Total Paid: €0.00 Pending Amount: €380.00

Figure 41 - Amounts coproducers view

In the same view, AMAP Management has another section related to the Amounts Payable to Producers, with information by producer on the total value of subscriptions, the amount already paid, and the amount pending payment to the producer.

Amounts to be Received by Producers
Balance Information User: John Doe (john.doe@example.com) Total Due: €150.00 Total Paid: €30.00 Pending Amount: €30.00 Producer: Quinta XPTO
User: Bruce Willis (bruce.willis@example.com) Total Due: €395.00 Total Paid: €0.00 Pending Amount: €0.00 Producer: Not specified
User: watala5709 (watala5709@ckuer.com) Total Due: €4.99 Total Paid: €0.00 Pending Amount: €0.00 Producer: Not specified

Figure 42 - Amounts producers view

5.6 US06

The story mapping describes how producers can view their subscribed products, modify delivered quantities based on production changes (both increases and decreases), and confirm updated delivery information. The mapping focuses on breaking the functionality into actionable steps and prioritizing development to ensure a smooth and efficient delivery update process.

This approach ensures producers can adapt their deliveries to current production conditions while maintaining accurate records for improved management and communication. The person responsible for documenting and structuring this user story was Luís Serapicos.

5.6.1 Story Mapping

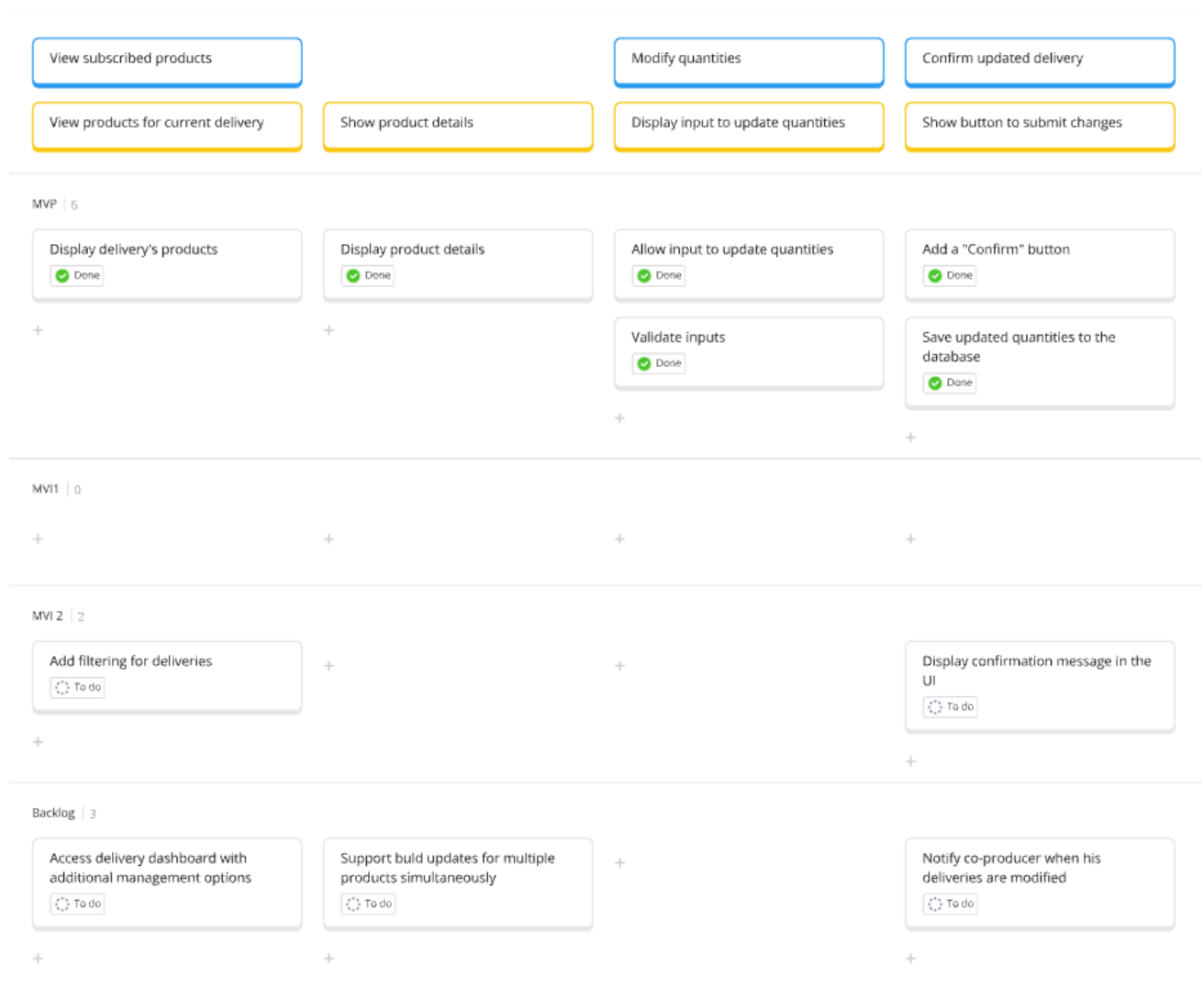


Figure 43 - Story Mapping for the user story US06: "producer should be able to update their orders quantities based on the production changes".

The story mapping for the user story, “As a producer, I want to be able to deliver subscribed products while modifying the quantities delivered to reflect changes (positive or negative) in production,” outlines a structured development plan focusing on three main activities: viewing subscribed products, modifying quantities, and confirming updated deliveries.

Viewing Subscribed Products: Producers can access a list of products and quantities up for delivery.

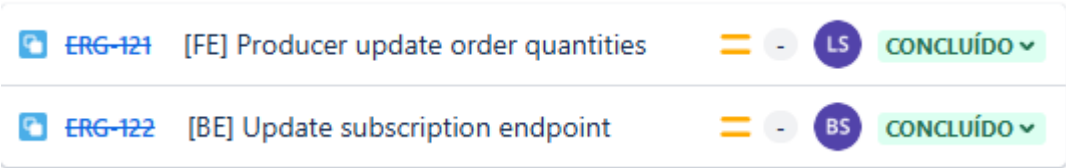
Modifying Quantities: Producers can adjust delivery quantities via an input interface. Backend validation ensures logical and accurate input, preventing invalid values.

Confirming Updated Delivery: Producers finalize modifications through a confirmation button. The system saves updated quantities to the database, maintaining accurate delivery records.

The MVI2 (Minimum Viable Increment 2) includes two tasks: "Add filtering for deliveries" and "Display confirmation message in the UI," both currently in the status TO DO. Future iterations will enhance usability with features like filtering options for deliveries, confirmation messages for successful updates, a delivery dashboard with advanced management options, bulk update functionality, and a notification system for co-producers.

5.6.2 Development and Prioritization

MVP: The MVP focuses on implementing the critical features necessary for producers to view and update delivery quantities. This includes functionalities that allow producers to access a list of subscribed products, modify delivery quantities based on production changes, validate these inputs, confirm updates, and save the changes to the database. These core features ensure that producers can manage their deliveries effectively with basic delivery modification capabilities.



 ERG-121	[FE] Producer update order quantities	 -  LS	CONCLUÍDO ✓
 ERG-122	[BE] Update subscription endpoint	 -  BS	CONCLUÍDO ✓

Figure 44 - Jira User Stories

MVI2: After delivering the MVP, additional features will be prioritized to enhance the user experience and efficiency. These include:

Add Filtering for Deliveries: Allow producers to sort and filter deliveries based on criteria such as date, status, or product type, making it easier to locate specific deliveries quickly.

Display Confirmation Message in the UI: Display confirmation messages in the user interface to provide immediate feedback when changes are successfully saved, reducing uncertainty and ensuring producers are aware that their updates were processed correctly.

This prioritization ensures that the most critical features are delivered first, providing immediate value to producers, while subsequent iterations focus on refining the user experience and adding advanced capabilities.

5.6.3 Process View

The graphical interfaces produced for this User Story are:

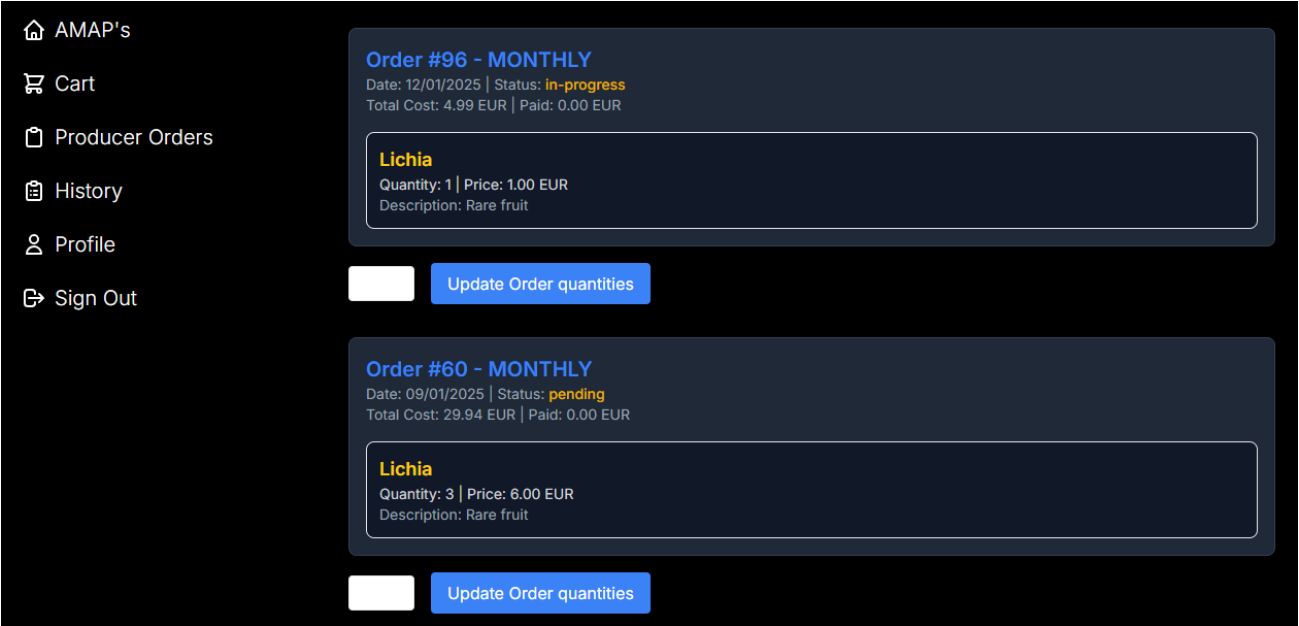


Figure 45 - This view "Producers Orders" shows the orders made to the products that the current Producer is able to produce.

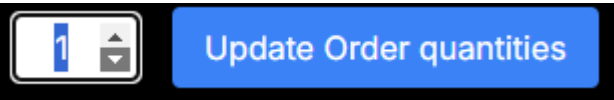


Figure 46 - The input validation only allows the user to update the quantity to non-negative values.

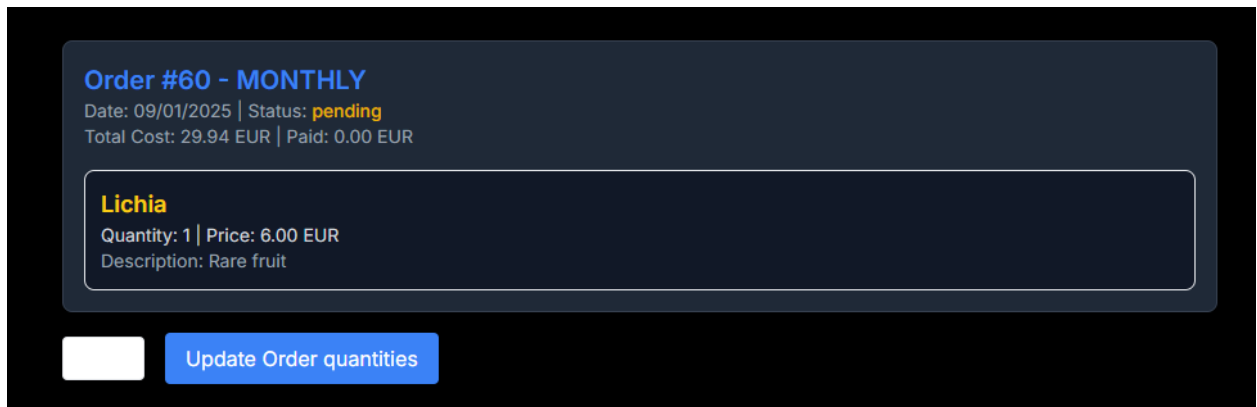


Figure 47 - Once the user triggers the "Update Order quantities" button the order quantity is updated with the correct value.

5.7 US07

5.7.1 Story Mapping

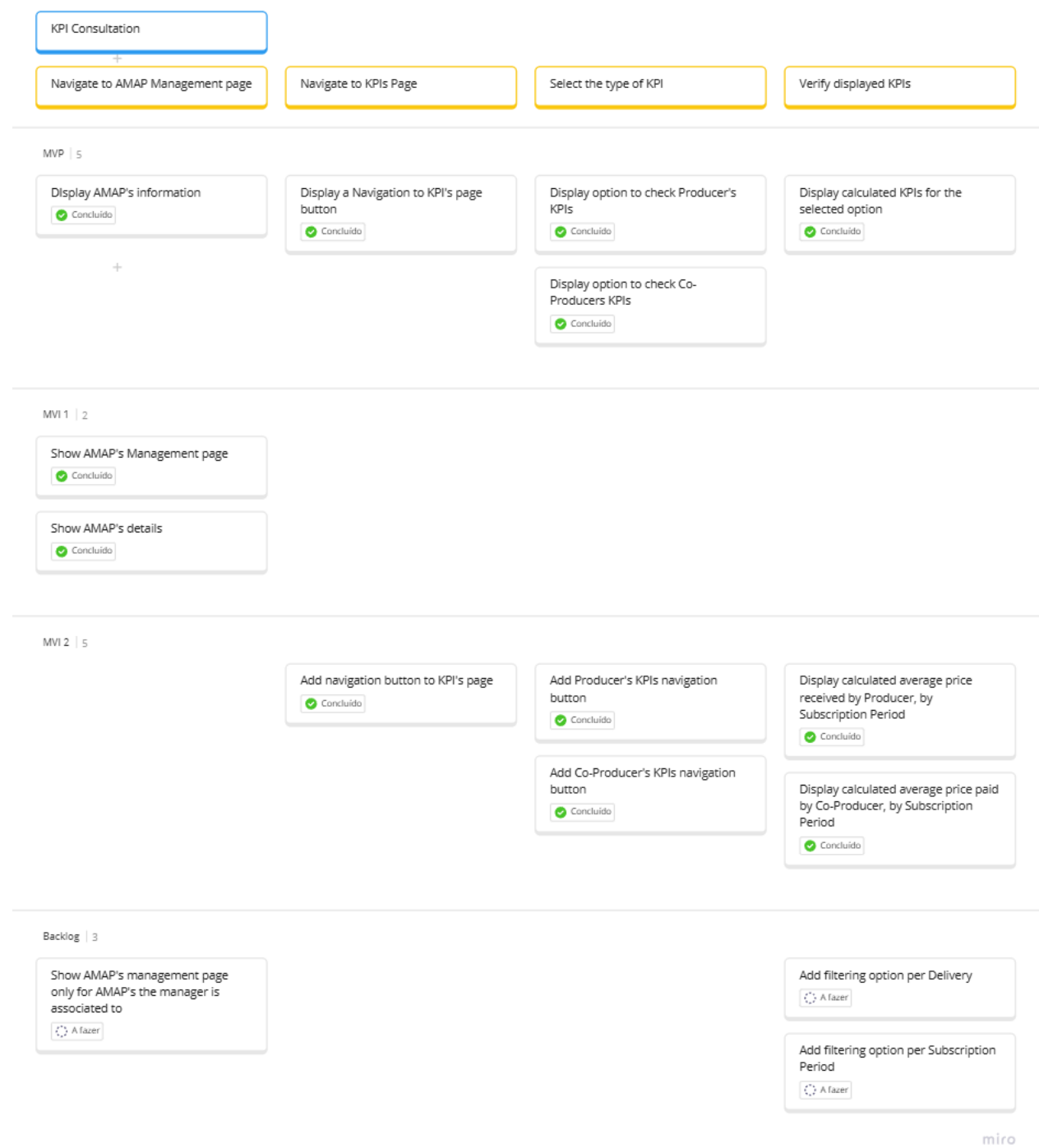


Figure 48 - Story mapping US07

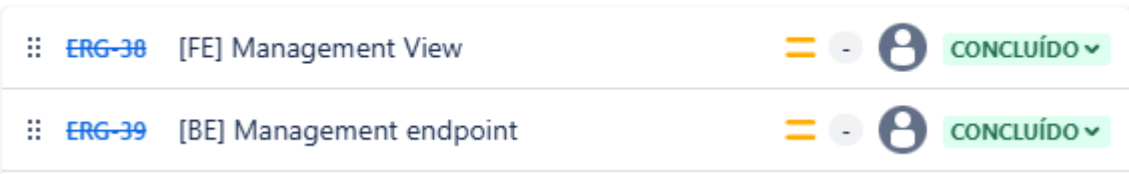
This Story Mapping outlines the functionalities and workflow for a feature focused on KPI Consultation within our AMAP management system. The primary purpose is to enable managers to access and analyze key performance indicators (KPIs) related to Producers and Co-Producers.

So, the Story Mapping shows that the Feature to be developed is designed to provide a structured, user-friendly process for navigating KPI-related information, selecting the relevant KPI types, and verifying the calculated data in a way it supports AMAP managers in monitoring and evaluating the general success between Producers and Co-Producers subscriptions.

5.7.2 Development and Prioritization

During Sprint 1, the focus for this User Story was to establish AMAP as an entity in the system and to develop a general management page for AMAP Managers. The first sprint primarily concentrated on building the project’s foundational structure and implementing general features that support various requirements, so developing the page was a priority as it provides access to key details and enables other functionalities.

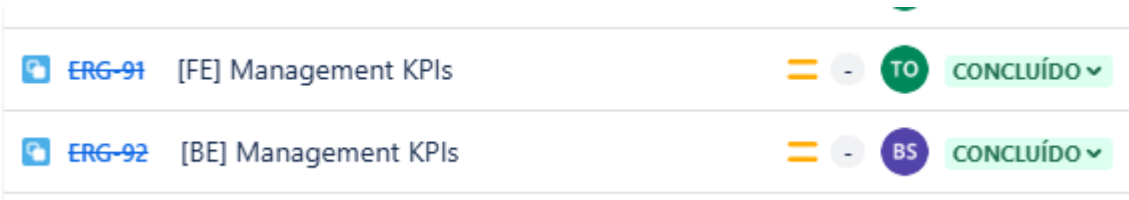
As part of this User Story, the tasks completed during Sprint 1 included creating the AMAP entity and enabling navigation to its main page.



ERG-38	[FE] Management View	=	-	👤	CONCLUÍDO ▾
ERG-39	[BE] Management endpoint	=	-	👤	CONCLUÍDO ▾

Figure 49 - Jira User Stories.

Then, on Sprint 2 the focus was to calculate the relevant KPIs to be displayed to the AMAP Managers when they access the management page. To do so, specific endpoints were added so the calculations would be made based if the Manager wants to check the average value spent by Co-Producers or received by Producers by Order and by Subscription period.



ERG-91	[FE] Management KPIs	=	-	👤	CONCLUÍDO ▾
ERG-92	[BE] Management KPIs	=	-	👤	CONCLUÍDO ▾

Figure 50 - Jira User Stories.

With these tasks completed, the majority of the User Story can be considered fulfilled. However, additional supporting tasks were added to the backlog as future enhancements. These tasks aim to refine the data displayed in the KPIs, making it more relevant and specific for AMAP Managers to evaluate the organization’s success in achieving its objectives. They will also ensure that Producers are meeting the required values associated with their Subscriptions.

 ERG-133 Add filters to the KPI's calculations	TAREFAS PEND...	
 ERG-134 Restrict data access to KPI's	TAREFAS PEND...	

Figure 51 - Jira User Stories.

Additional types of KPIs should be considered for future backlogs, as discussions with AMAP representatives emphasized the importance of having metrics to track the organization's goals. It is also likely that, in the future, each AMAP may need the ability to define its own KPIs, as different AMAPs could prioritize varying metrics. However, we have decided not to add these KPI-related tasks to the backlog at this time, as the specific requirements for such customizations have not yet been defined.

5.7.3 Process View

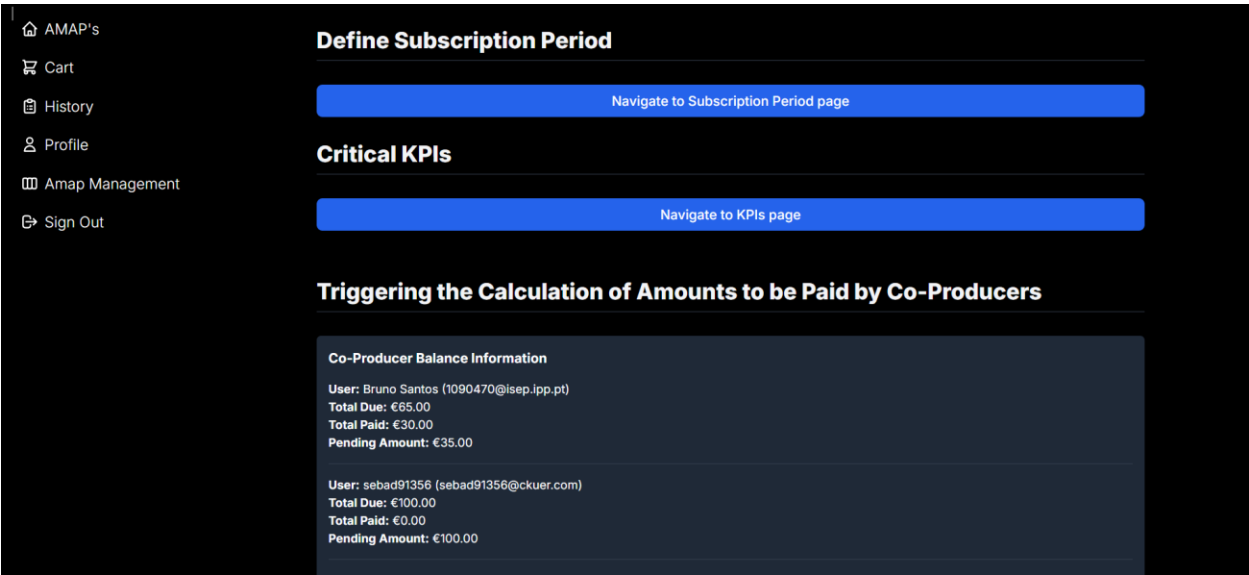


Figure 52 - Amap management view.

For the AMAP Manager to check the AMAP’s KPIs, it should first navigate to the AMAP Management home page, and then click on the “Navigate to KPIs page” button.



Figure 53 - KPIs menu view.

It will redirect the Manager to a landing page for the KPIs in order to trigger the metrics calculations for the Producer or for the Co-Producer. Both can be seen by selecting one of the “Producer KPIs” or “Co-Producer KPIs” buttons.

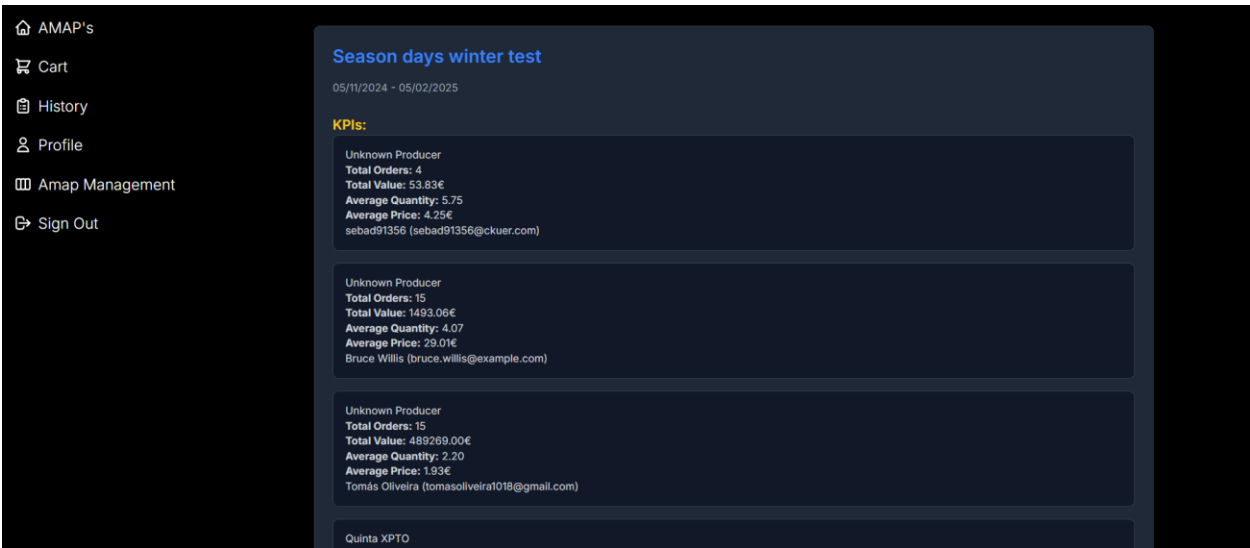


Figure 54 - KPIs view.

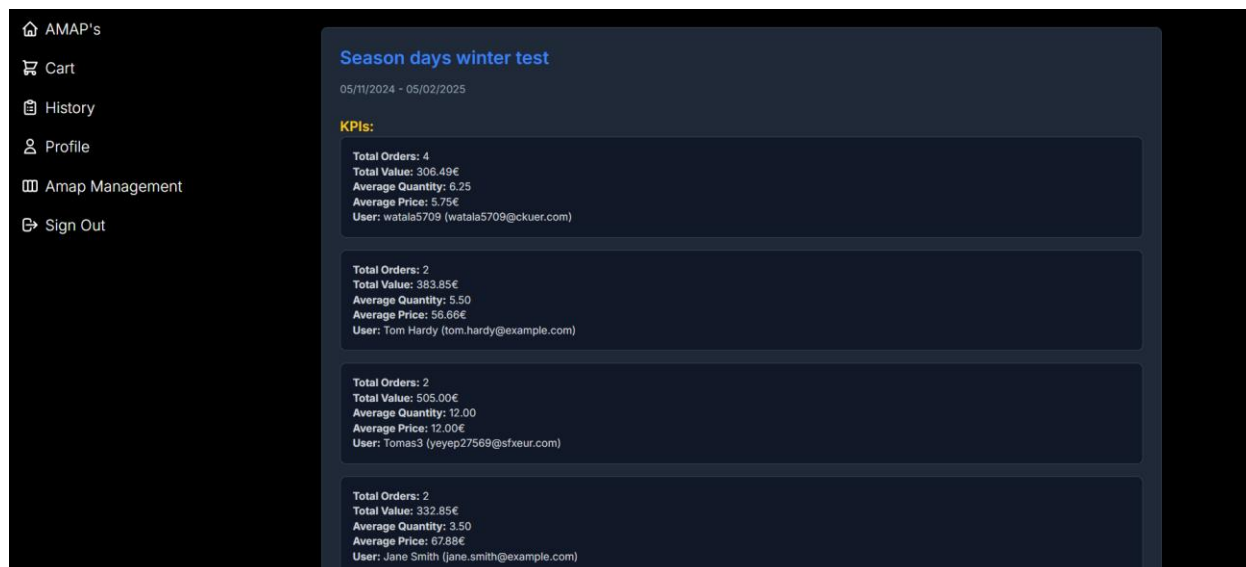


Figure 55 - KPIs view.

For both options selected, a list will be displayed showing the average values and total orders by Subscription period. For Producer KPIs, this will include the average values and total orders received. For Co-producer KPIs, the list will display the average values paid and the total orders made. This distinction ensures that the relevant metrics are clearly presented based on the role of the user that should be analyzed.

5.8 US08

The user story is centered on a calculation and reconciliation process that determines the financial differences between the amounts paid by consumers and the value of the products to deliver. Once these differences are identified, they are automatically recorded in the account balances of the co-producers. This functionality ensures that any overpayments or underpayments are properly accounted for and reflected in the co-producers' financial records.

The person responsible for documenting and structuring this user story was Luís Serapicos.

5.8.1 Story Mapping

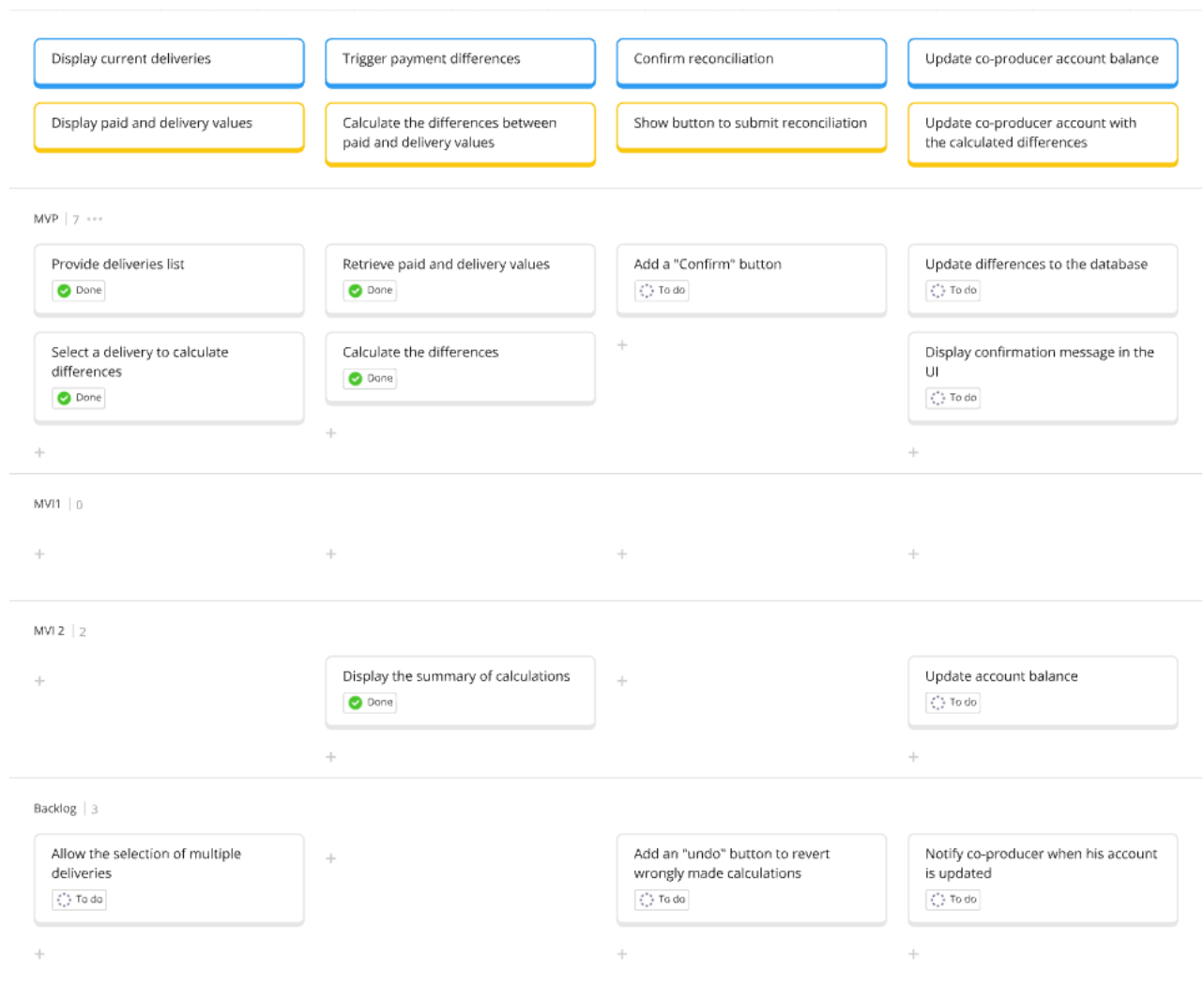
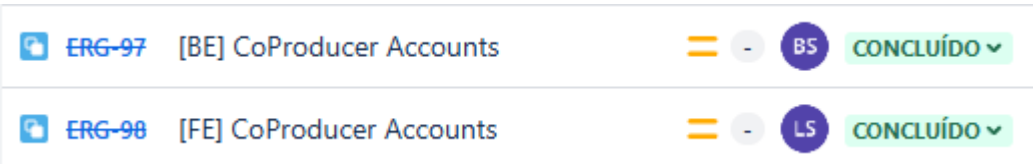


Figure 56 - Story Mapping for the user story US08: "AMAP Managers should be able to trigger paid values and delivery values calculations and place the difference on co-producer's accounts".

The story mapping for this user story describes the implementation of a feature to help AMAP managers reconcile differences between paid amounts and delivery values, ensuring accurate updates to co-producers' account balances. It begins with the ability to display current deliveries. This step provides managers with a comprehensive view of the list of deliveries, including the paid and delivery values. This visibility forms the foundation for managers to assess discrepancies and proceed with reconciliation.

5.8.2 Development and Prioritization

MVP: The MVP focuses on delivering the core functionality necessary for AMAP managers to reconcile differences between paid amounts and delivery values. The objective is to provide a functional, minimal system that addresses the reconciliation needs without additional complexity.



 ERG-97	[BE] CoProducer Accounts	 - 	CONCLUIDO 
 ERG-98	[FE] CoProducer Accounts	 - 	CONCLUIDO 

Figure 57 - Jira User Stories.

On the backend, the system retrieves delivery data (including paid and delivery values) from the database and implements a calculation engine to determine the differences. These services form the backbone of the reconciliation process. On the frontend, the interface is designed to be simple, allowing managers to view a list of deliveries, select one for reconciliation, and trigger the calculation of discrepancies. The goal is to ensure ease of use while providing the essential features required for accurate reconciliations.

The following tasks were left to do on a next sprint, because we considered that showing the values to the co-producers had higher priority since there is no use to have a feature to trigger the values and not show them (the co-producer would not know about them):

- Adding a "Confirm" button: This ensures that reconciliation actions are intentional and verified, reducing errors.
- Automatically updating differences in the database: This step integrates calculated differences directly into the database, automating updates to co-producer account balances.
- Providing UI confirmation messages: Real-time feedback informs managers that reconciliation actions have been completed successfully, improving transparency and user satisfaction.

MVIs: MVIs are iterative improvements to the MVP that aim to enhance the system's functionality and user experience over time. They provide incremental value while ensuring stability and scalability.

MVI1: This phase is intentionally left empty to allow the team to focus on perfecting the MVP.

MV12: The second increment introduces key enhancements to the reconciliation process:

Displaying a summary of calculated differences: Managers can review a detailed breakdown of discrepancies, improving decision-making accuracy.

5.8.3 Process View

The graphical interfaces produced for this User Story are:

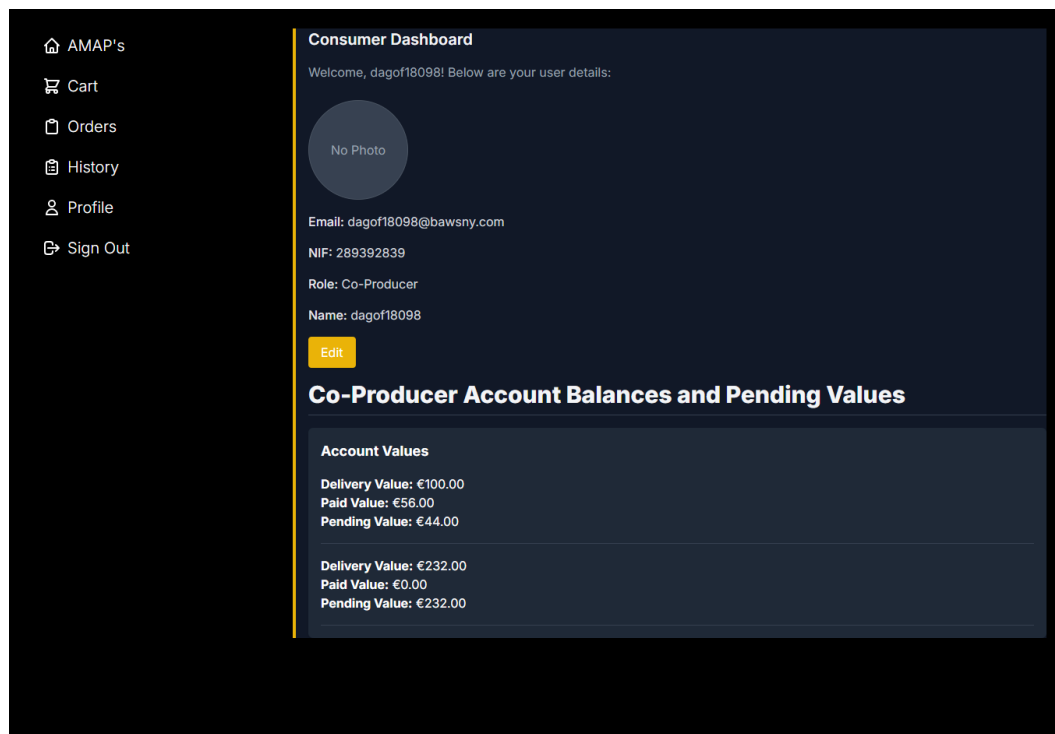


Figure 58 - Co-producer profile view where they can, beside other features, see the values that they need to pay/receive from pending orders.

5.9 US09

“Como Produtor quero poder criar um cabaz composto por produtos simples que podem também podem existir oferta de produtos isoladamente”

Responsible: Guilherme Cota

This use case focuses on creating a basket of products and is a set of simple products. The products can exist and be offered outside of the basket context.

5.9.1 Story Mapping

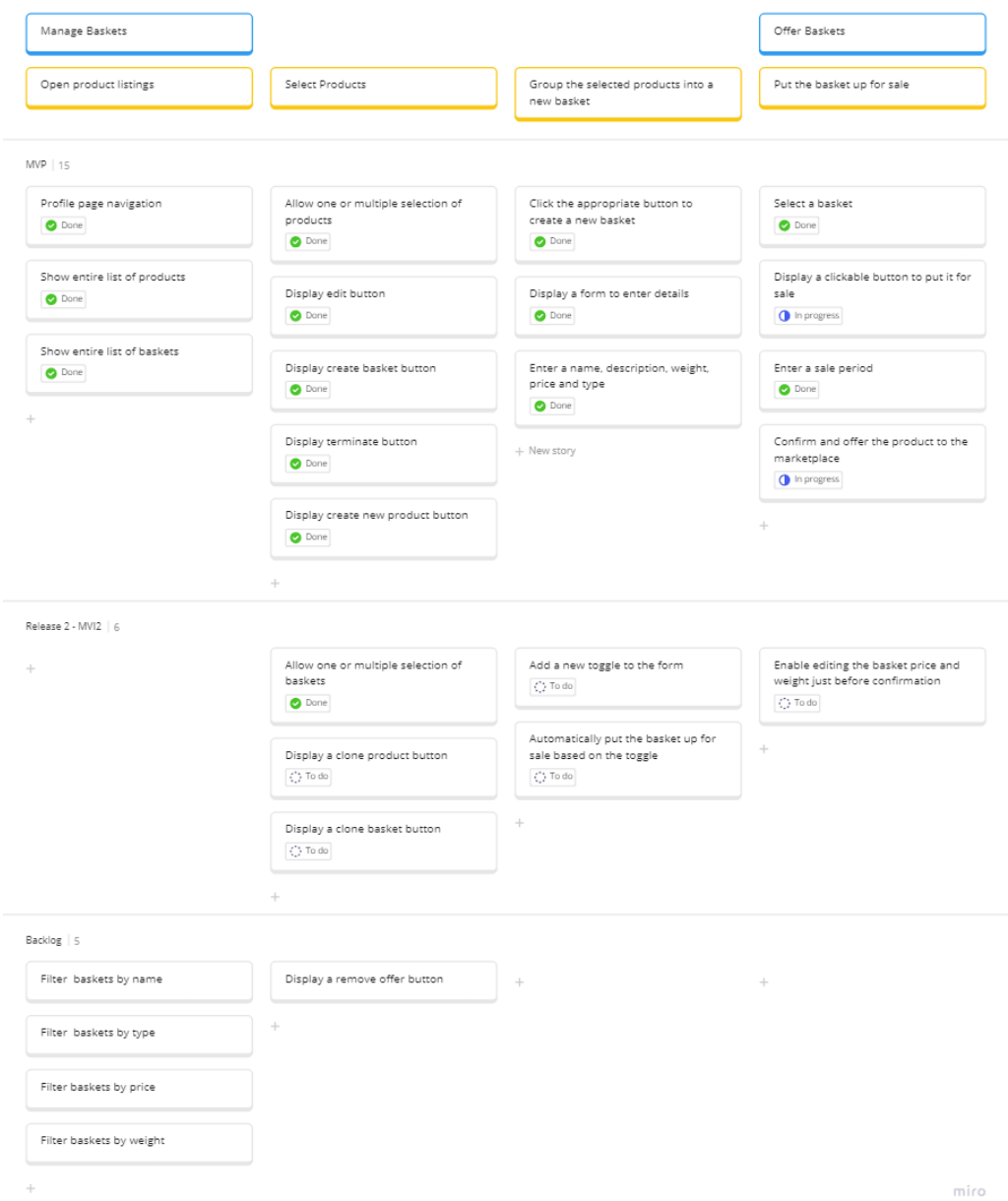


Figure 59 - Story mapping US09

The story mapping shows how a producer can create and manage baskets in the system
The process is broken down into four main steps:

"Open product listings"

View the complete list of available products, access the product listing interface and see all existing baskets.

"Select Products"

Choose one or multiple products to include in a basket. Include buttons in the interface for creating new products, creating new baskets, editing products and removing products

"Group the selected products into a new basket"

Click to create a new basket, Fill out a form with basket details such as name, description, weight, price and type

"Put the basket up for sale"

Select the specific basket, define a sale period, confirm and publish to the marketplace

The story mapping includes a Release 2 section with planned improvements:

Multiple basket selection capability, a cloning functionality for products and baskets, a toggle feature for automatic basket publishing and the ability to edit basket pricing and weight just before confirmation

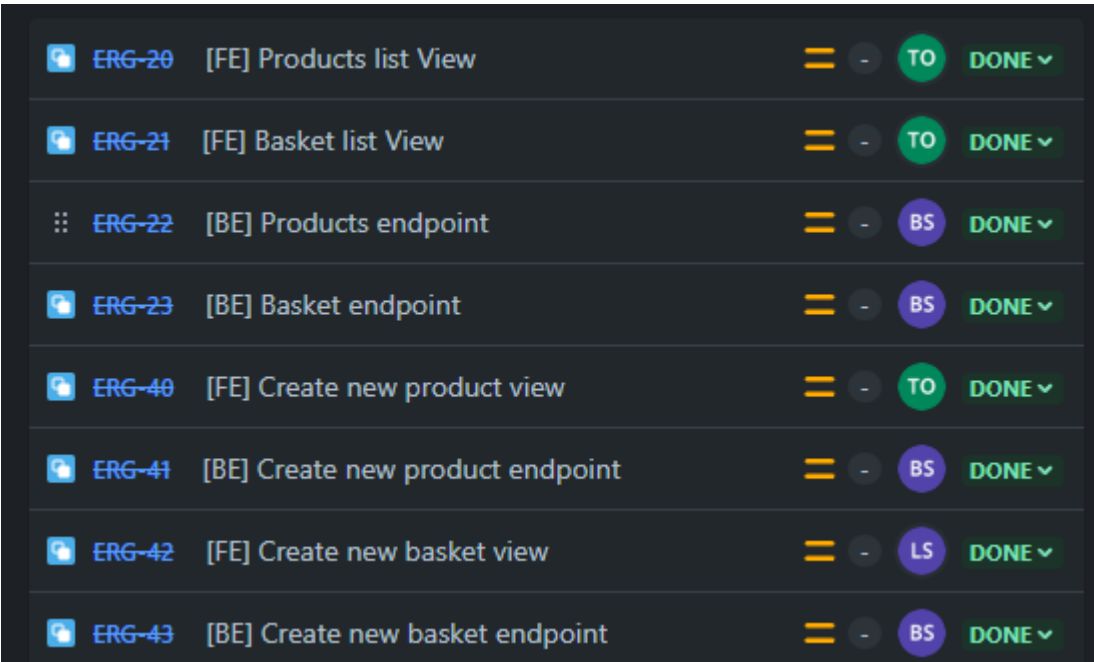
The story mapping also has a Backlog section with more improvements which were not part of the releases. Such as enabling multiple filtering options, by name, type, price and weight and displaying a remove offer button.

Each step has clear status indicators (Done/In Progress/To Do) to show the current development state of each feature.

5.9.2 Development and Prioritization

We prioritized these tasks in this way because having the profile implemented in the UI would allow us to implement the basket and product CRUD behaviors like creating, editing and removing. We wanted to have a place where the user could see all his information and manage it. This means that based on the role, the profile would have more or less

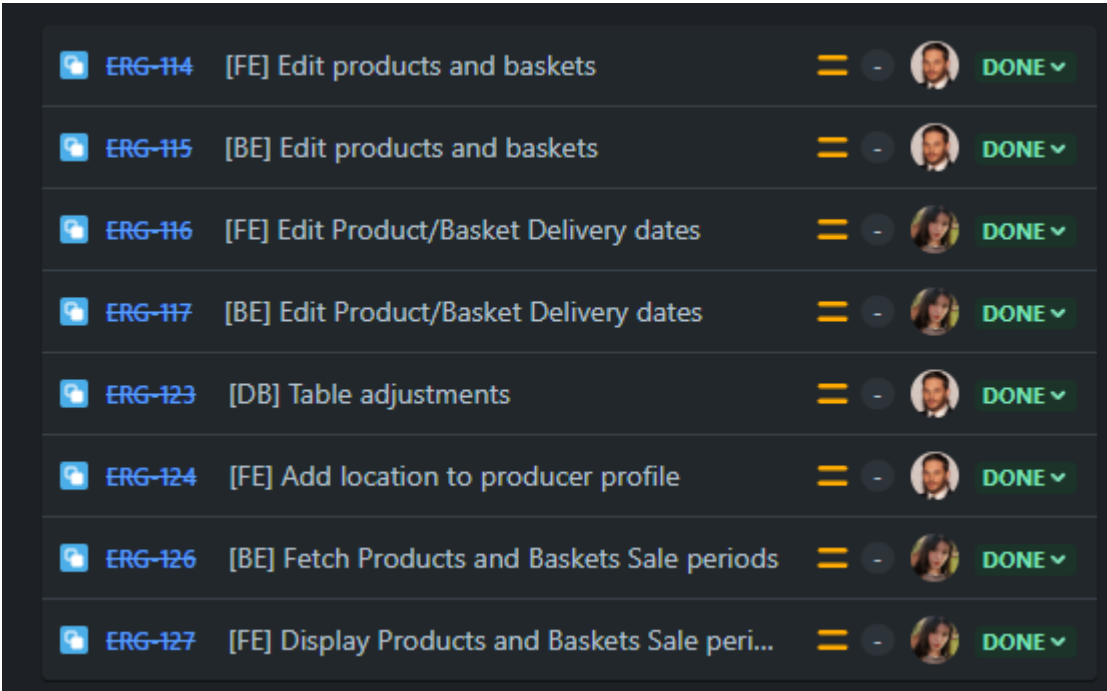
functionality. In the case of the producer, it has more functionality because it shows the baskets and the products listings.

A screenshot of a Jira User Stories table. The table has a dark background with light text. It contains eight rows of user stories. Each row starts with a blue icon (a person with a plus sign) followed by a story ID in blue, a description in white, and then a series of icons: two yellow horizontal bars, a minus sign, a colored circle with a letter, and a green 'DONE' button with a dropdown arrow. The stories are: ERG-20 [FE] Products list View (TO), ERG-21 [FE] Basket list View (TO), ERG-22 [BE] Products endpoint (BS), ERG-23 [BE] Basket endpoint (BS), ERG-40 [FE] Create new product view (TO), ERG-41 [BE] Create new product endpoint (BS), ERG-42 [FE] Create new basket view (LS), and ERG-43 [BE] Create new basket endpoint (BS).

	ERG-20	[FE] Products list View			-		DONE ▾
	ERG-21	[FE] Basket list View			-		DONE ▾
	ERG-22	[BE] Products endpoint			-		DONE ▾
	ERG-23	[BE] Basket endpoint			-		DONE ▾
	ERG-40	[FE] Create new product view			-		DONE ▾
	ERG-41	[BE] Create new product endpoint			-		DONE ▾
	ERG-42	[FE] Create new basket view			-		DONE ▾
	ERG-43	[BE] Create new basket endpoint			-		DONE ▾

Figure 60 - Jira User Stories.

After developing the endpoints and views necessary for the creation of baskets we also implemented the profile page which allows the selection of baskets and products. It also allows some off-topic functionality due to the context of it being a profile page.

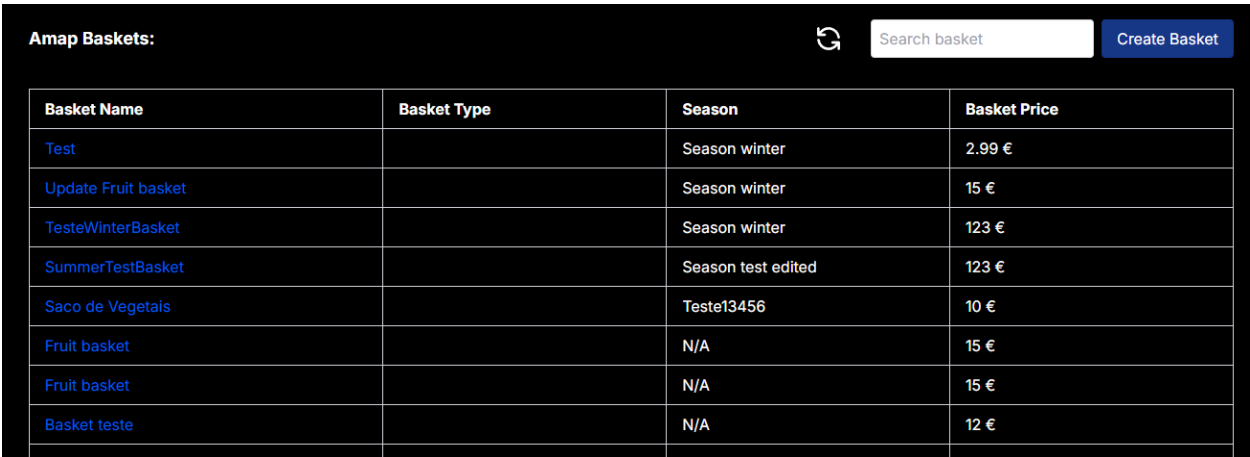


	ERG-114	[FE] Edit products and baskets	 		DONE ▾
	ERG-115	[BE] Edit products and baskets	 		DONE ▾
	ERG-116	[FE] Edit Product/Basket Delivery dates	 		DONE ▾
	ERG-117	[BE] Edit Product/Basket Delivery dates	 		DONE ▾
	ERG-123	[DB] Table adjustments	 		DONE ▾
	ERG-124	[FE] Add location to producer profile	 		DONE ▾
	ERG-126	[BE] Fetch Products and Baskets Sale periods	 		DONE ▾
	ERG-127	[FE] Display Products and Baskets Sale peri...	 		DONE ▾

Figure 61 - Jira User Stories.

5.9.3 Process View

In this view we can select the create basket button



Amap Baskets:					Search basket	<button>Create Basket</button>
Basket Name	Basket Type	Season	Basket Price			
Test		Season winter	2.99 €			
Update Fruit basket		Season winter	15 €			
TesteWinterBasket		Season winter	123 €			
SummerTestBasket		Season test edited	123 €			
Saco de Vegetais		Teste13456	10 €			
Fruit basket		N/A	15 €			
Fruit basket		N/A	15 €			
Basket teste		N/A	12 €			

Figure 62 - Amap catalog view.

Afterwards a modal is opened and prompts the user to insert the necessary data, and lets us select the sale period

Create Basket

Name
Basket of products

Description
products

Select	Season Name	Start Date	End Date
☐	Season days winter test	11/5/2024	2/5/2025
☐	Create Season Test	11/5/2024	2/5/2025
☐	Test Season	1/11/2025	1/23/2025
☐	Sigma Season	1/11/2025	5/14/2025
☐	123	1/11/2025	1/31/2025
☐	SeasonTest123	1/25/2025	3/28/2025
☒	Teste13456	1/16/2025	2/14/2025
☐	Season winter	11/5/2024	2/5/2025
☐	Season test edited	11/5/2024	2/5/2025

Selected Season's Delivery Dates

Price
1.50

Weight
0.15

Products
SummerTest WinterTest Teste x | v

Photo
No photo selected
Select Photo

Create Basket FPS N/A

Figure 63 - Create basket view.

After clicking "create basket" the basket persisted in the database.

We can then go to our profile and see our product and basket listings, and we can also see that the products are individual and can be offered outside of a basket.

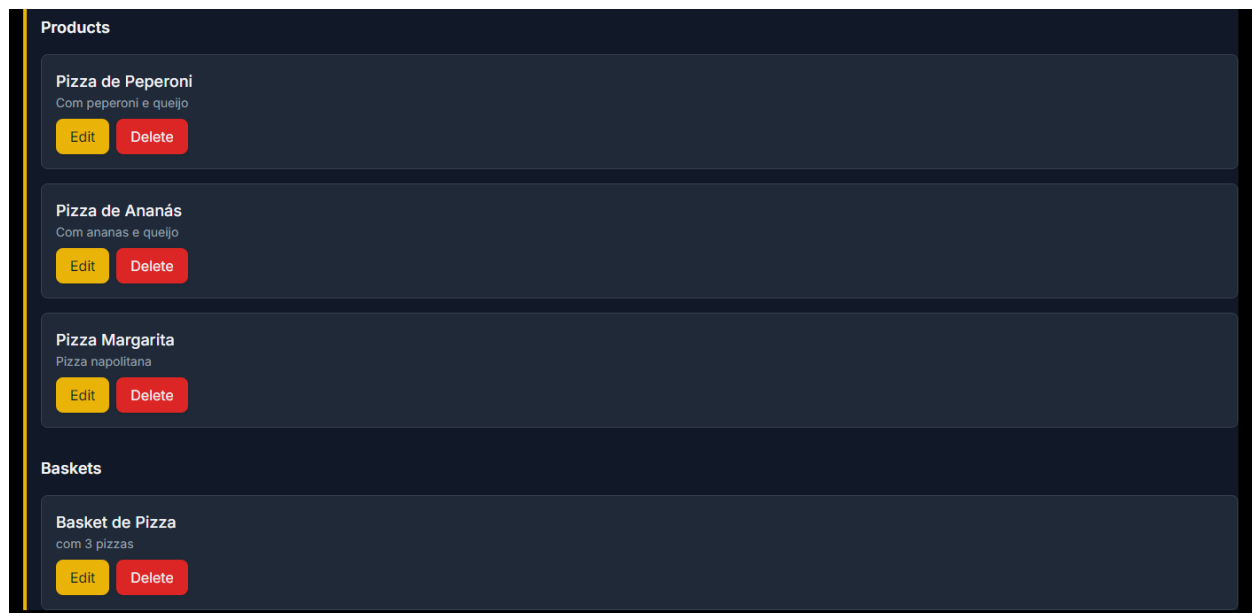


Figure 64 - Product and products producer profile view.

6. Backbone

In order to develop the solution, it was necessary to create the entire project structure, which was then used to implement all the necessary user stories, which was divided as follows:

- Frontend structure
- Backend structure
- Database (tables and relations)

6.1 Frontend

React was used to develop the solution on the frontend, and at this point it was necessary to create the entire structure of the web portal so that it could host all the functionalities. The structure was created according to the selected technology, but also following good practices.

All the necessary structure, middleware, communication services with the API were implemented. In order to communicate with the API, a Bearer token was implemented to ensure the security and authentication of the user's requests.

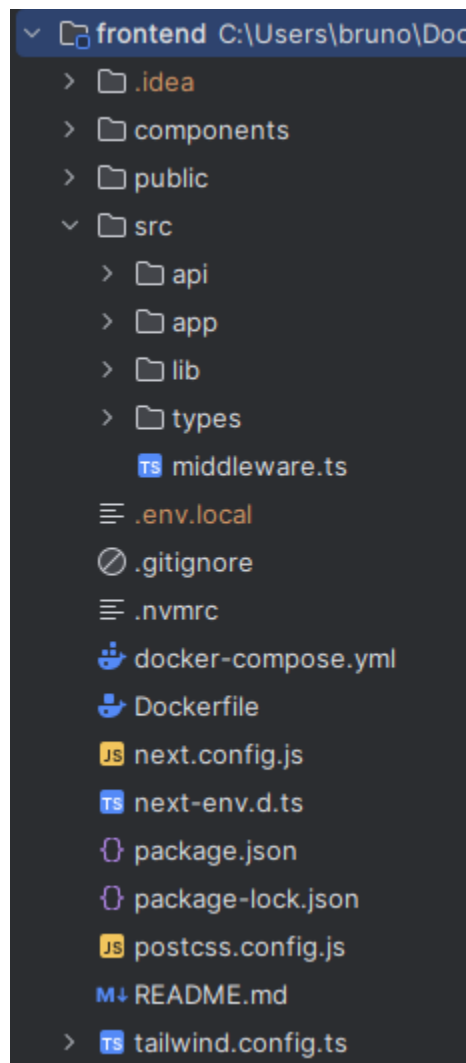


Figure 65 - FE structure

6.1.1 Middleware

We created a middleware using the `@supabase/auth-helpers-nextjs` library to manage authentication inside our Next.js application. The middleware first initializes a response object with `NextResponse.next()` to allow the request to proceed by default. A key aspect of its functionality is handling public and protected routes.

```
const res = NextResponse.next();

const publicUrls = ['/reset', '/login', '/register', '/confirm'];
```

It checks if the requested URL matches a predefined list of public paths, such as `/reset`, `/login`, `/register`, or `/confirm`.

If the request targets one of these public routes, it is allowed to continue without any further checks, ensuring users can access specific pages without authentication.

For all other routes, the middleware integrates with Supabase by creating a middleware client that links the request and response objects. This client is used to retrieve the current user session through the `supabase.auth.getSession()` method.

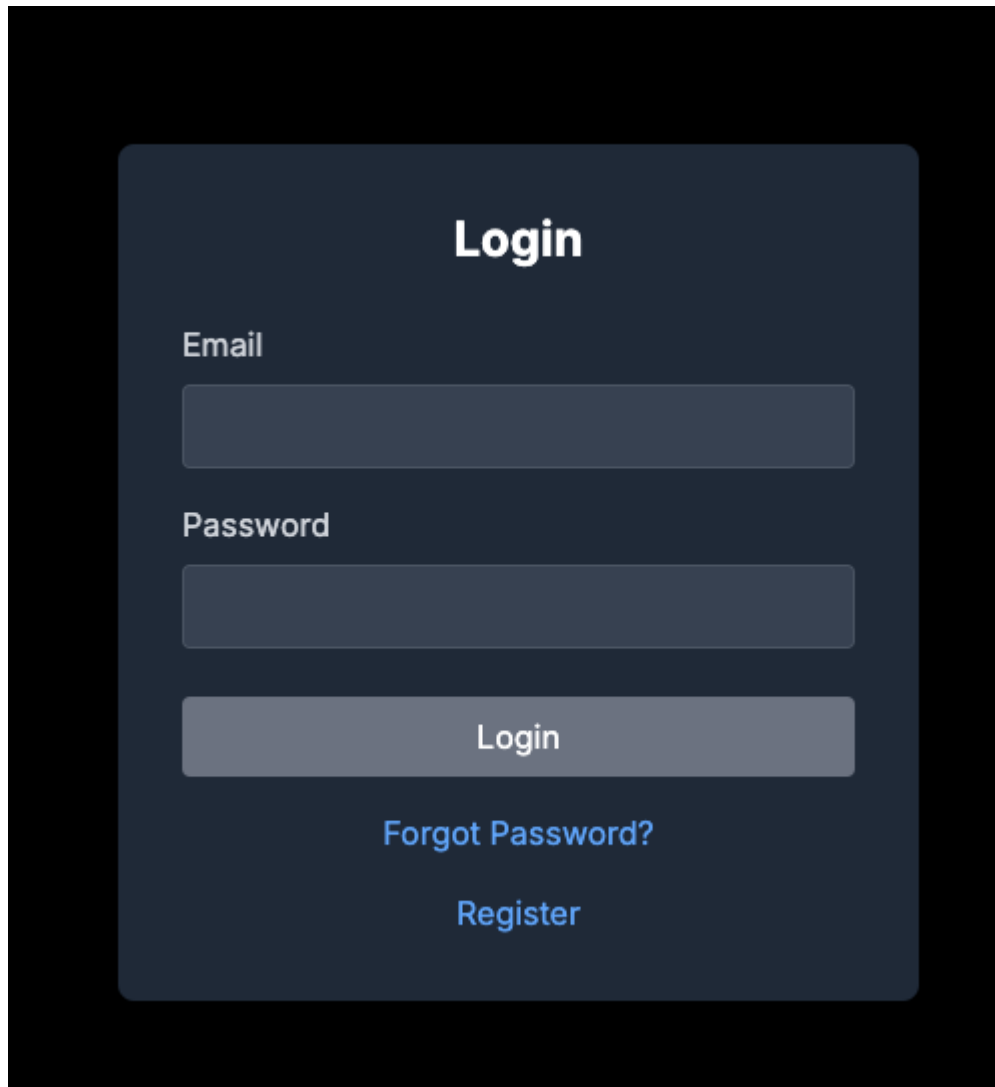
If no session is found, indicating that the user is not authenticated, the middleware redirects the user to the `/login` page. However, if a valid session exists, the request is processed as intended.

The middleware configuration includes a matcher that defines the routes it should handle. It excludes API endpoints, static assets, and other system files, ensuring these routes are unaffected by the authentication logic. This implementation effectively secures protected resources by allowing only authenticated users to access them while maintaining unrestricted access to public pages.

6.2 Public Pages

6.2.1 Login Page

A login page has been developed that utilizes Supabase for both authentication and data management within a Next.js application. This interface enables users to log in using their email and password, reset their passwords when necessary, and create new accounts. It features a responsive user interface that adjusts according to user interactions, providing distinct modes for logging in and recovering passwords.



The image shows a dark-themed login page. At the top, the word "Login" is displayed in a large, bold, white font. Below this, there are two input fields: "Email" and "Password", both with white text labels and dark gray input boxes. Under the password field is a "Login" button with white text on a dark gray background. Below the button are two links: "Forgot Password?" and "Register", both in a light blue color. The entire login card is centered on a black background.

Figure 66- Login page

The authentication process checks user credentials through Supabase's authentication service. Upon successful login, the user's session is confirmed, and their information is synchronized with a specific database table. This synchronization process ensures that user metadata, including role, name, and tax identification number (NIF), is either updated or inserted into the database, with email serving as the unique identifier to prevent duplicates. In the event of errors during the login or synchronization processes, users receive appropriate notifications to guide them.

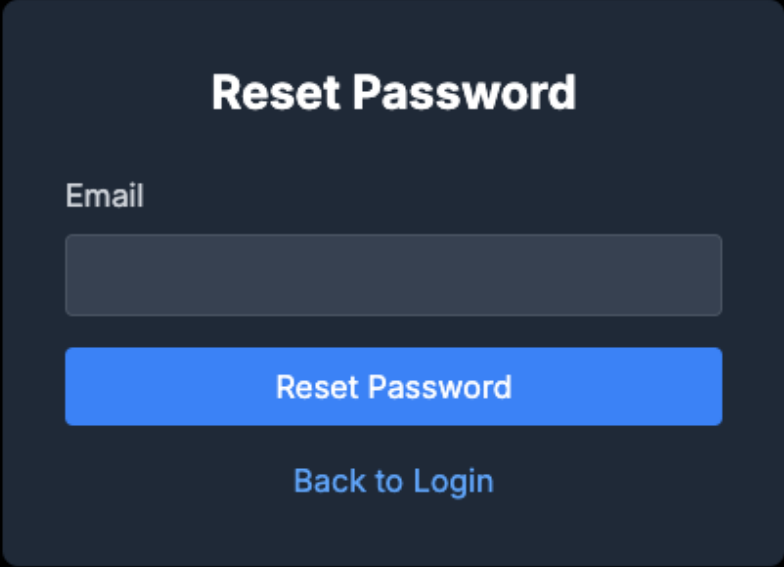
The image shows a 'Reset Password' form on a dark background. The form is a dark gray rounded rectangle. At the top, the title 'Reset Password' is centered in white bold text. Below the title, the label 'Email' is positioned to the left of a light gray rectangular input field. Underneath the input field, there is a solid blue button with the text 'Reset Password' in white. At the bottom of the form, the text 'Back to Login' is centered in a light blue color.

Figure 67 - Reset password page

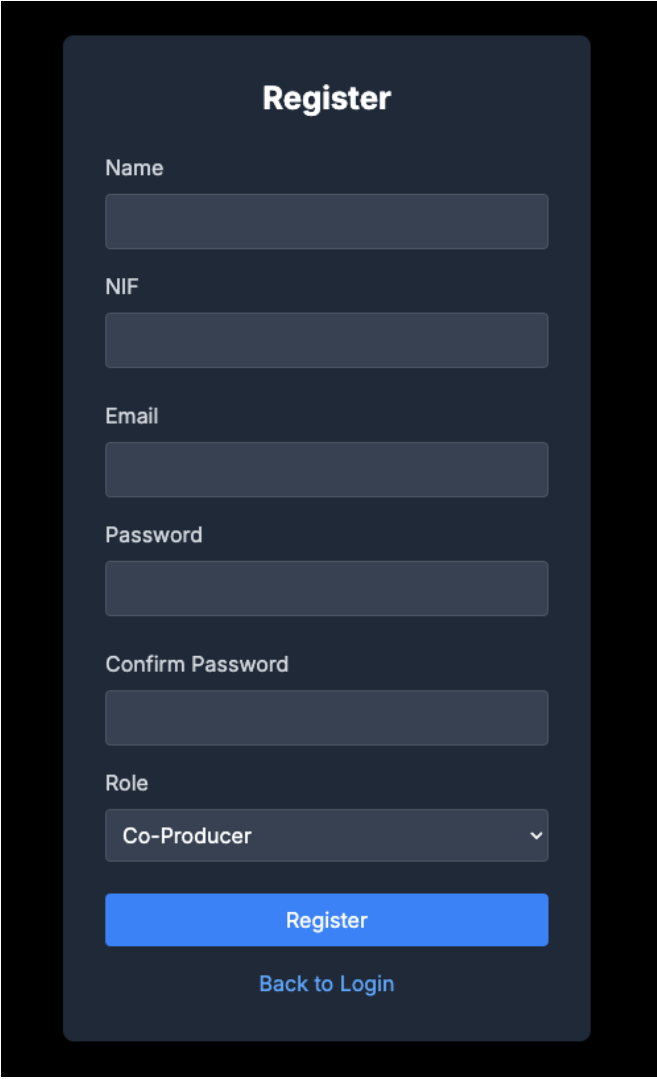
For those requiring password recovery, the page includes a dedicated mode that facilitates this process. It sends a password reset email via Supabase, which contains a link directing users to a reset page. The interface also offers navigation options between login, password recovery, and account registration, thereby ensuring a seamless experience tailored to various user requirements.

To improve usability, the login button is disabled when mandatory fields are not filled out, thereby preventing invalid submissions. Clear error messages and feedback are provided to

assist users in understanding and resolving any issues encountered during the login process. This login page is meticulously designed to deliver a secure and user-friendly authentication experience, effectively utilizing Supabase's functionalities for both authentication and database management.

6.2.2 Register Page

We developed a registration interface that enables users to create an account while ensuring the validation of their inputs and the secure management of user information through Supabase. This interface features a form where users can enter details, including their name, email address, password, password confirmation, tax identification number (NIF), and designated role. The design of the interface aims to facilitate a seamless registration experience, providing clear feedback regarding any issues encountered during the submission of the form.



The image shows a registration form titled "Register" on a dark background. The form contains the following fields and elements:

- Name:** A text input field.
- NIF:** A text input field.
- Email:** A text input field.
- Password:** A text input field.
- Confirm Password:** A text input field.
- Role:** A dropdown menu with "Co-Producer" selected and a downward arrow.
- Register:** A prominent blue button.
- Back to Login:** A text link below the Register button.

Figure 68 - Register page

The registration procedure integrates input validation mechanisms to confirm that the data provided is both accurate and complete prior to the user's registration attempt. This validation encompasses checks for mandatory fields such as name and email, verification of a valid NIF consisting of exactly nine digits, and assurance that the password adheres to specified length requirements and corresponds with the confirmation input. These validation measures are instrumental in preventing the submission of invalid or incomplete data, with user-friendly error messages presented whenever discrepancies are identified.

Upon successful validation of the form, the registration process advances by securely transmitting the user's information to Supabase via the signUp method. In addition to the email and password, the registration request incorporates supplementary metadata, including the user's role, name, and NIF. If the registration is completed successfully, the user is redirected to the main page of the application. Should any errors arise during the registration process, a clear message is provided to assist users in identifying and rectifying the issue.

Furthermore, the page offers flexibility by allowing users to choose their role from predefined categories such as "Producer" or "Co-Producer." A "Back to Login" button is also included to enhance navigation, enabling users to transition effortlessly between the login and registration processes. In summary, this registration interface prioritizes data integrity and security while delivering a user-friendly experience for account creation.

6.2.3 Reset Page

A password reset page has been developed to facilitate the secure updating of user account passwords. This page features an intuitive and accessible interface, allowing users to input their new password and confirm it for accuracy.

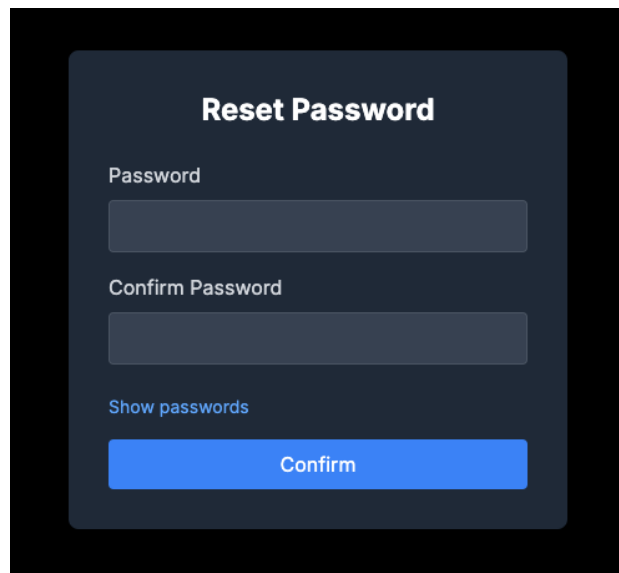
The image shows a 'Reset Password' form with a dark blue background. The form has a title 'Reset Password' at the top. Below the title are two input fields: 'Password' and 'Confirm Password'. Below these fields is a link 'Show passwords' in blue text. At the bottom of the form is a blue button labeled 'Confirm'.

Figure 69 - Reset page

The password reset procedure requires users to enter their new password in two separate fields: one designated for the password and another for confirmation. This dual-entry system is intended to minimize errors by ensuring that users accurately input their desired password twice. In instances where the passwords do not match, users receive immediate feedback, which prevents the submission of incorrect information. Once the passwords are confirmed to match, the new password is securely updated through Supabase's user management API. Following a successful update, users are redirected to the application's main page.

To further improve usability, the page incorporates a toggle feature that allows users to reveal or conceal their passwords while typing. This functionality aids in reducing input errors by enabling users to verify their entries prior to submission. Moreover, the overall design and layout of the page are crafted to ensure that users can navigate the password reset process with ease and clarity.

6.2.4 Confirm

A confirmation page has been developed to authenticate user email addresses during the registration process, ensuring that accounts are activated only after successful validation. This page facilitates secure email verification and offers users clear feedback regarding the outcome of the process.

Upon accessing the confirmation page, the system automatically retrieves parameters from the URL, including the verification token, email address, and action type (signup). These parameters are crucial for the email validation process. If any necessary information is absent or incorrect, the page promptly notifies the user that the verification link is invalid.

When the parameters are deemed valid, the page communicates with Supabase to verify the one-time password (OTP) linked to the email. A successful verification results in an update to the user's account status, confirming their registration. Subsequently, the page presents a success message and redirects the user to the login page after a brief delay, ensuring a seamless transition to finalize the signup process. In the event of any verification errors, the user receives a clear message outlining the issue.

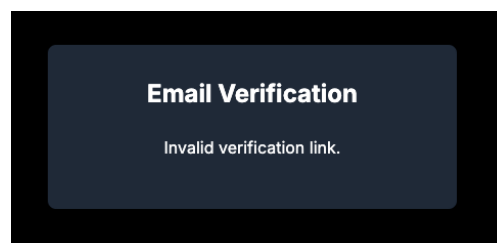


Figure 70 - Email verification

This confirmation page is meticulously designed to provide a secure and user-centric email verification experience. By validating user information and delivering immediate feedback, it significantly enhances the security and dependability of the account creation process.

6.3 Backend

For the backend, a Node.js API was implemented using the Express framework in order to ensure the entire database connection, providing the necessary endpoints for the solution to work.

Middleware was implemented in the API to ensure user authentication, as well as validating their permissions to use certain endpoints. An error handler was also implemented to respond accordingly. As mentioned above, a Bearer token is used to validate access to the API, as well as validating your authentication in the service.

All orders follow code structuring and good software practices, and each route has its own associated controller to manage the order. Each controller for making requests to the database has an associated service for direct actions to the DB.

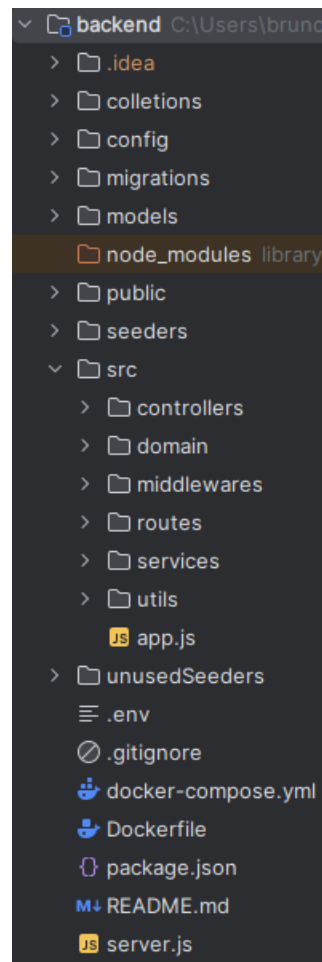


Figure 71 - BE structure

6.3.1 Middleware

The following middleware was implemented in the backend:

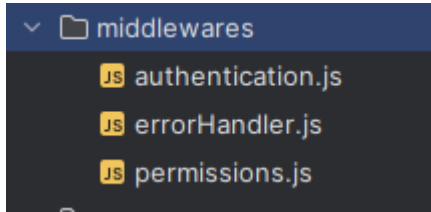


Figure 72 - Middleware structure

- **Authentication:** In this middleware, the bearer token is validated in every request if it exists and if it is valid. As well as through the token, we validate that the session is valid and is not expired in the authentication service used, which is Supabase. In this middleware, once validation and authentication is done with Supabase, a user session is also created, improving performance and validations during the API.
- **Error Handler:** It is a middleware used to handle errors in a centralized way, as well as ensuring that the API returns coherent information in accordance with the error detected. It also deals with unexpected errors, always returning an error to the user.
- **Permissions:** In this middleware and according to the user's role, it is validated whether they have permissions for certain endpoints made available. There are some endpoints and/or actions that are limited according to the user's role.

Example:

```
{  
  "message": "Invalid or expired token",  
  "details": "invalid JWT: unable to parse or verify signature, token has invalid claims: token is expired"  
}
```

6.3.2 Controllers / Services

The API was structured to follow good practices and separate the business areas. It is therefore structured according to the routes structure, with each group having its own controller, allowing requests to be managed and processed. Each group also has a service, allowing requests to the database.

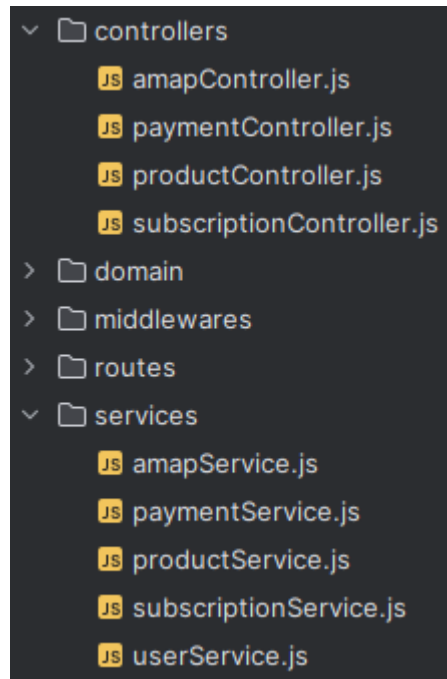


Figure 73 - Controllers/service structure

In the services for DB requests, as already mentioned, Sequelize was used, which will be detailed in the database topic. Sequelize allows queries to be made in a simple way, and was used in all the services.

```
const productDetails = await Product.findAll({
  attributes: ['id', 'name', 'description', 'type', 'price', 'quantity', 'photoUrl'],
  where: {
    id: productID,
  },
  include: [
    {
      model: Producer,
      attributes: ['id', 'businessName'],
      required: true,
      include: [
        {
          model: User,
          attributes: ['id', 'email', 'nif'],
        },
        {
          model: Certificate,
          attributes: ['id', 'name', 'issuingAuthority', 'issueDate', 'expirationDate'],
        },
      ],
    },
  ],
});
```

6.3.3 Logger

To improve the functioning of the API, a logger system was implemented, allowing requests to be recorded. Another advantage of the logger is that, at this stage of development, it allows rapid analysis of the path and action of each request, facilitating development and possible corrections and improvements.

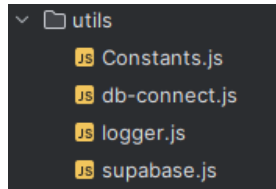


Figure 74 - Utils structure

Example:

```
info: Retrieve CoProducer Payment list
info: Incoming request: GET /payment
info: Fetching data user:
Executing (default): SELECT "email", "role", "AMAPId" FROM "Users" AS "User" WHERE "User"."email" = '1090470@isep.ipp.pt';
info: Retrieved user data: {"email":"1090470@isep.ipp.pt","role":"Co-Producer","AMAPId":4}
info: Request getOrderList
info: Requesting CoProducer Payment list
```

6.3.4 Routes

In order to organize the structure of the routes used, they were grouped according to the basis of the structure and functions of the platform, so the following groups were created:

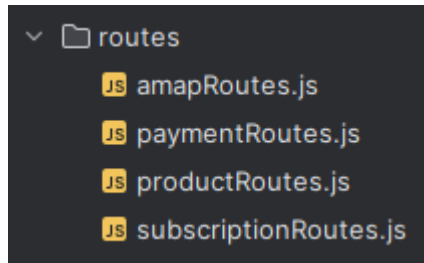


Figure 75 - Routes structure

We are going to detail all the available groups, as well as the endpoints available in each of the groups, but for details of each endpoint and the arguments required, the technical documentation and arguments required for each of them are available via swagger.

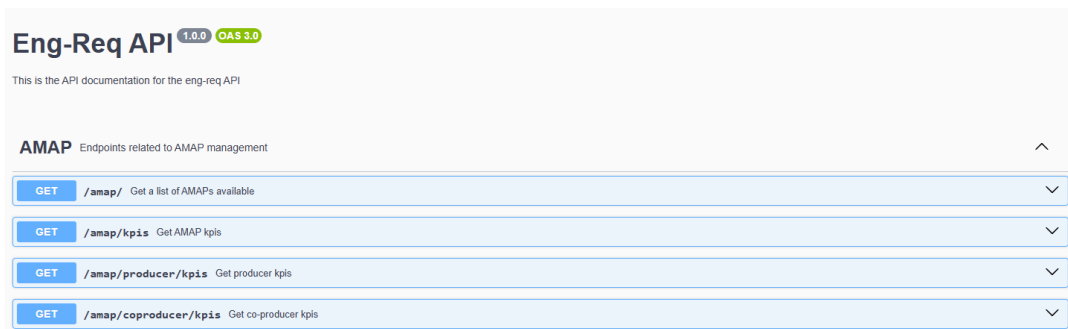


Figure 76 - Swagger endpoint example

6.4 Database

To house all the information in the solution and respond to needs, a structure of tables and relations was created and projected in SRS. To implement the database, we used Supabase, creating a PostgreSQL database.

To automatically map our code to the tables, we made use of an ORM like Sequelize.

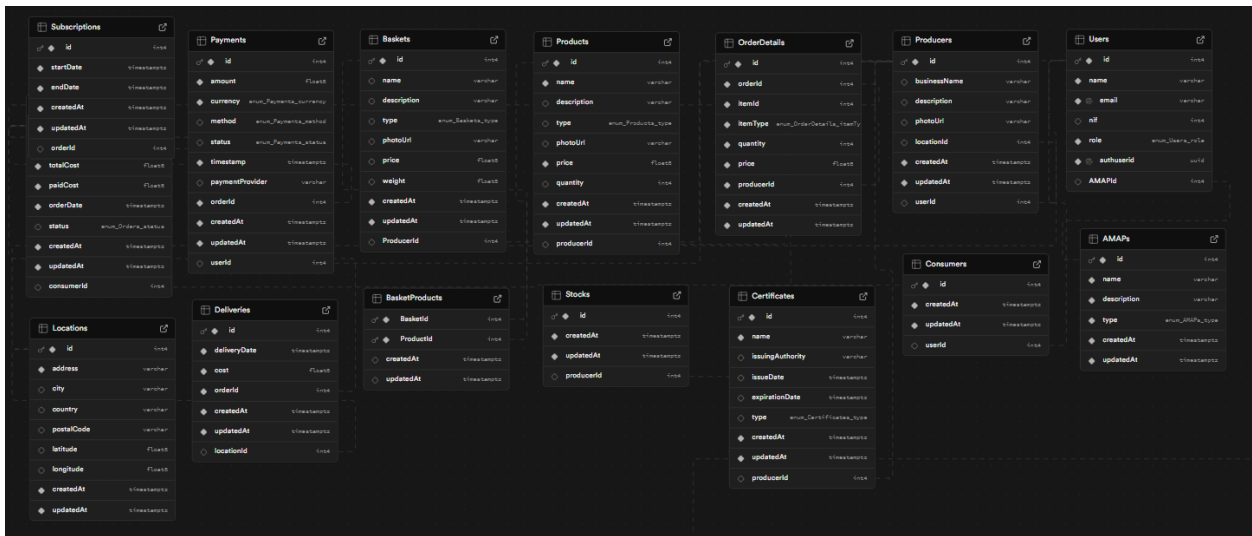


Figure 77 - Database tables

Sequelize is a popular Node.js ORM (Object-Relational Mapping) library that simplifies database interactions by providing a structured way to work with relational databases such as PostgreSQL, MySQL, MariaDB, SQLite, and Microsoft SQL Server. It allows developers to define models for database tables using JavaScript or TypeScript and includes powerful features like model relationships, query building, validation, and migrations. With Sequelize, you can perform CRUD operations (Create, Read, Update, Delete) using a syntax that abstracts SQL queries, making it easier to manage and maintain the database layer of your application.

Sequelize supports advanced ORM functionalities such as associations (e.g., one-to-one, one-to-many, many-to-many relationships), eager loading, transaction management, and schema synchronization. It integrates seamlessly with modern frameworks like Express.js, enabling developers to build scalable applications with ease. By reducing the need for raw SQL, Sequelize allows developers to focus on application logic while ensuring that database interactions are efficient and secure.

Code Examples:

```
const sequelize : Sequelize = new Sequelize(  
  process.env.DB_NAME,  
  process.env.DB_USER,  
  process.env.DB_PASSWORD,  
  {  
    host: process.env.DB_HOST,  
    port: process.env.DB_PORT,  
    dialect: 'postgres'  
    // dialectOptions: {  
    //   ssl: {  
    //     require: true,  
    //     rejectUnauthorized: false, // Required for Supabase  
    //   },  
    // },  
  }  
);  
  
// Test the connection  
sequelize.authenticate() // Guilherme  
  .then(() : any | void => console.log('Database connected...'))  
  .catch(err => console.error('Error connecting to the database:', err));
```

This code initializes a Sequelize instance to connect to a PostgreSQL database (supabase) using connection details stored in an environment file (.env)

```
// Sync models  
sequelize.sync({ options: { force: false } }) // Use `force: true` to drop and recreate tables (only for development)  
  .then(() : any | void => console.log('Database synced'))  
  .catch(err => console.error('Error syncing the database:', err));
```

This code snippet uses Sequelize to **synchronize the database schema** with the defined models in our application.

```
class User extends Model {} Show usages ⓘ Guilherme
User.init({ attributes: {
  name: { type: DataTypes.STRING, allowNull: false },
  email: { type: DataTypes.STRING, allowNull: false, unique: true },
  nif: { type: DataTypes.INTEGER },
  role: { type: DataTypes.ENUM( values: 'Producer', 'Co-Producer', 'Admin', 'AMAP Admin'), allowNull: false },
  // Foreign key to Supabase auth.users
  authuserid: { type: DataTypes.UUID, allowNull: false, unique: true },
}, options: {
  sequelize,
  timestamps: false,
  modelName: 'User' });
```

This code defines a Sequelize model for a User entity, mapping it to a database table and specifying its structure, validations, and behavior.

```

User.hasOne(Consumer, options: { foreignKey: 'userId' });
User.hasOne(Producer, options: { foreignKey: 'userId' });
User.belongsTo(AMAP, options: { foreignKey: 'AMAPId' });
User.hasMany(Payment, options: { foreignKey: 'userId' });

Stock.belongsTo(Producer, options: { foreignKey: 'producerId' });

Product.belongsTo(Producer, options: { foreignKey: 'producerId' });

Producer.hasMany(Product, options: { foreignKey: 'producerId' });
Producer.belongsTo(User, options: { foreignKey: 'userId' });
Producer.hasOne(Stock, options: { foreignKey: 'producerId' });
Producer.hasMany(Certificate, options: { foreignKey: 'producerId' });

Payment.belongsTo(User, options: { foreignKey: 'userId' });

Order.belongsTo(Consumer, options: { foreignKey: 'consumerId' });
Order.hasMany(Delivery, options: { foreignKey: 'orderId' });

Order.hasMany(Payment, options: { foreignKey: 'orderId' });
Payment.belongsTo(Order, options: { foreignKey: 'orderId' });

Producer.belongsTo(Location, options: { foreignKey: 'locationId' });
Location.hasOne(Producer, options: { foreignKey: 'locationId' });

Order.hasOne(Subscriptions, options: { foreignKey: 'orderId' });
Subscriptions.belongsTo(Order, options: { foreignKey: 'orderId' });

Location.hasOne(Delivery, options: { foreignKey: 'locationId' });

```

This code defines and maps the relationships between various Sequelize models, setting up associations such as `hasOne`, `hasMany`, `belongsTo`, and `belongsToMany`. It effectively specifies how different database tables are connected and how those relationships should be enforced.

6.4.1 Seeders and Migrations

To have data readily available every time we had to do a hard refresh of our database, we made use of sequelize seeders. These seeders will populate each table, insert data that connects using queries and bulkInserts. Although not necessary, we also implemented migrations that specify the tables to be created. These migrations haven't been necessary because everytime we run the project, sequelize is initializing our schema and making sure it is correctly synced with the database.

Code

Examples:

```
const consumers = await queryInterface.sequelize.query(
  `SELECT id FROM "Consumers";`,
  {
    type: QueryTypes.SELECT,
  }
);

const currentDate : Date = new Date();
const orders : any[] = [];

// Generate orders (single purchase, monthly, weekly)
consumers.forEach((consumer) : void => {
  // Single purchase order
  orders.push({
    consumerId: consumer.id,
    periodType: periodType.single_purchase,
    totalCost: Math.random() * 100 + 50, // Random total cost between 50 and 150
    paidCost: Math.random() * 50 + 20, // Random paid cost between 20 and 70
    orderDate: currentDate,
    status: status.pending,
    createdAt: currentDate,
    updatedAt: currentDate,
  });

  // Monthly order
  orders.push({
    consumerId: consumer.id,
    periodType: periodType.monthly,
    totalCost: Math.random() * 200 + 100, // Random total cost between 100 and 300
    paidCost: Math.random() * 150 + 50, // Random paid cost between 50 and 200
```

The script generates orders for consumers based on specific types (single purchase, monthly, weekly) and inserts these orders into the Orders table.

The script retrieves non-single-purchase orders and prepares to create corresponding Subscriptions (though this is currently commented out because a trigger is being used to create the subscriptions).

6.4.2 Database Triggers

In our database we also have defined some triggers that will run on insert, in specific tables.

We have 3 triggers in total. The 1st trigger is for Consumer replication, whenever a User with the role “Co-Producer” is inserted, we create a Consumer and reference the userId.

```
CREATE OR REPLACE FUNCTION create_consumer_on_user_insert()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if the role is 'Producer'
    IF NEW.role = 'Co-Producer' THEN
        -- Insert a new Producer record linked to the User's ID
        INSERT INTO public."Consumers" ("userId", "createdAt", "updatedAt")
        VALUES (NEW.id, NOW(), NOW());
    END IF;

    -- Return the newly inserted row
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

The 2nd trigger is for Producer replication, whenever a User with role “Producer” is inserted, we create a Producer and reference the userId. This ensures coesion.

```
CREATE OR REPLACE FUNCTION create_producer_on_user_insert()
RETURNS TRIGGER AS $$
BEGIN
    -- Check if the role is 'Producer'
    IF NEW.role = 'Producer' THEN
        -- Insert a new Producer record linked to the User's ID
        INSERT INTO public."Producers" ("userId", "createdAt", "updatedAt")
        VALUES (NEW.id, NOW(), NOW());
    END IF;

    -- Return the newly inserted row
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

We also have a trigger to create subscriptions. Given that the order holds all the information necessary, the subscription table just holds very simple data, like a start and end date, and a reference to the order it was triggered from.

```
CREATE OR REPLACE FUNCTION create_subscription_on_order_insert()
RETURNS TRIGGER AS $$
DECLARE
    start_date DATE;
    end_date DATE;
BEGIN
    -- Set the startDate to the current date
    start_date := CURRENT_DATE;

    -- Calculate the endDate based on the periodType
    IF NEW."periodType" = 'weekly' THEN
        end_date := start_date + INTERVAL '7 days';
    ELSIF NEW."periodType" = 'monthly' THEN
        end_date := start_date + INTERVAL '30 days';
    ELSE
        -- Handle other period types or set end_date to NULL for unsupported cases
        end_date := NULL;
    END IF;

    -- Insert a new subscription only if the periodType is not 'single purchase'
    IF NEW."periodType" != 'single purchase' THEN
        INSERT INTO public."Subscriptions" ("orderId", "createdAt", "updatedAt", "startDate", "endDate")
        VALUES (NEW.id, NOW(), NOW(), start_date, end_date);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

The script above essentially checks to see that the order type is not "single purchase" meaning that the Consumer chose either a weekly or monthly delivery plan. (these plans can be extended in the future).

If the user chooses a weekly period, then we will create a subscription starting from the current time and ending a week later (7 days). The same applies for Monthly, we add 30 days to the startDate.

Afterwards we insert the subscription in the table.

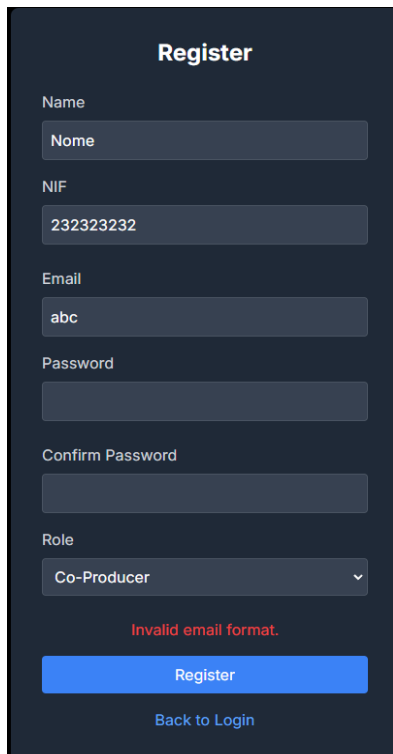
7. Non-functional requirements

The non-functional requirements implemented in our project align closely with those outlined in the SRS document, with additional feedback incorporated through stakeholder interactions.

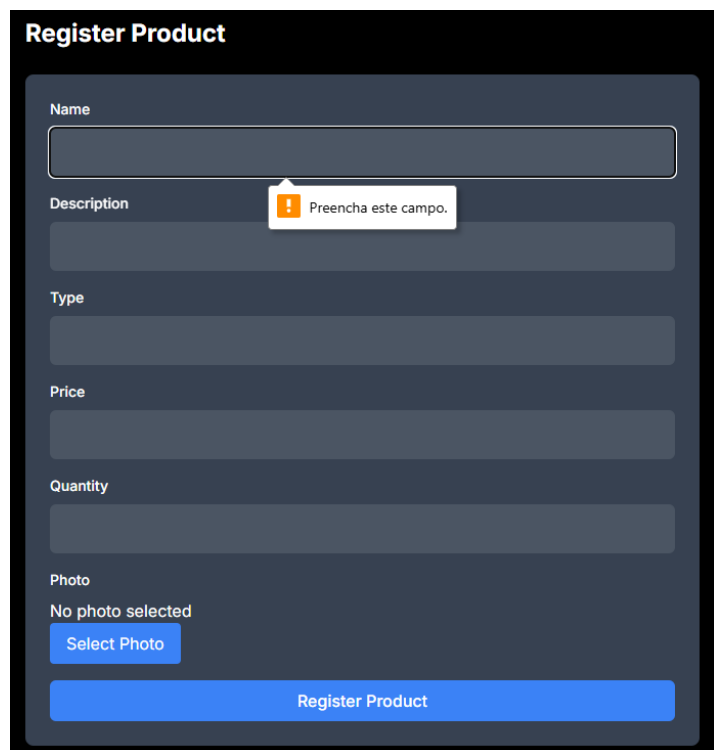
For usability, ARIA attributes were defined to label inputs, buttons, and forms, enhancing accessibility for users with disabilities. Additionally, ARIA attributes like `aria-hidden`, `aria-live`, and `aria-sort` were incorporated to improve dynamic content updates and provide a clearer structure for assistive technologies, ensuring a more inclusive user experience.

```
<input class="p-2 border border-gray-600 rounded bg-gray-700 text-white" aria-label="Enter your Name" type="text" value name="name">
<th class="border border-gray-300 px-4 py-2 text-left" aria-sort="descending">⋮</th>
<div aria-live="assertive">Subscribe to Product</div>
```

Furthermore, user-friendly error handling is incorporated across various forms, including the registration and product/basket addition processes.



The 'Register' form is a dark-themed interface with white text. It includes input fields for Name (placeholder 'Nome'), NIF (placeholder '232323232'), Email (placeholder 'abc'), Password, and Confirm Password. A dropdown menu for Role is set to 'Co-Producer'. A red error message 'Invalid email format.' is displayed below the email field. At the bottom, there is a blue 'Register' button and a 'Back to Login' link.



The 'Register Product' form is a dark-themed interface with white text. It includes input fields for Name, Description (with a tooltip 'Preencha este campo.'), Type, Price, Quantity, and Photo. The Photo field shows 'No photo selected' and a blue 'Select Photo' button. At the bottom, there is a blue 'Register Product' button.

Cart

Product Name: Laranja

Subscription Type:

Monthly

Quantity:

0

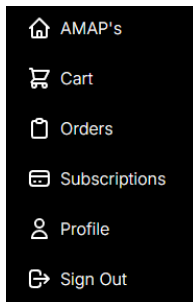
!

O valor tem de ser superior ou igual a 1.

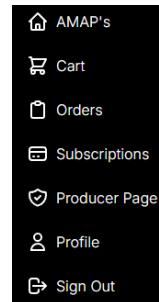
Figure 78 - Cart view.

Regarding security, Supabase is the backbone of our authentication system, offering features such as hashed password storage, and role-based access control (RBAC) to restrict user actions based on roles. This approach is mirrored both in the frontend (for controlling access to pages and HTML elements) and in the backend (for securing API endpoints).

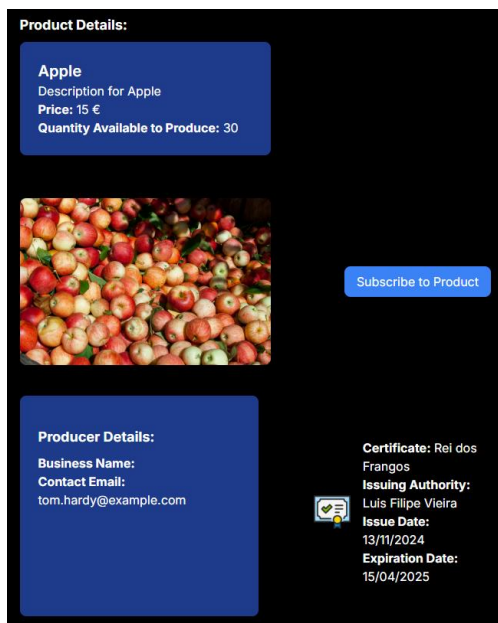
Co-Producer available pages:



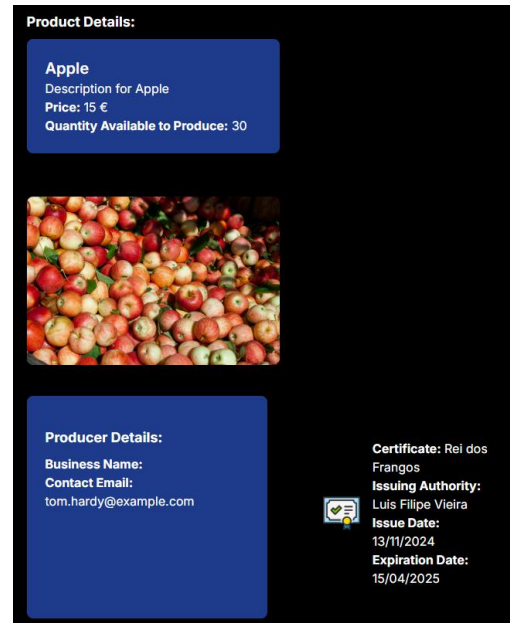
Producer available pages:



Co-Producer can subscribe to items:



Producer can't subscribe:



For safety, the implemented strict input validation also helps to prevent malicious attacks such as SQL injection and cross-site scripting (XSS), with specific field restrictions like email formatting and NIF validation. We also keep track of *createdAt* and *updatedAt* timestamps, every time an object is persisted onto the database we update the *updatedAt* field or the *createdAt* if we insert something new.

The application meets compatibility standards by being accessible on the latest versions of Chrome, Firefox, Safari, and Edge, and includes a responsive design that adjusts to various screen sizes, ensuring a consistent user experience across desktop and mobile platforms.

The project also follows RESTful principles for the backend, ensuring that all API endpoints are stateless and adhere to standard HTTP methods. This approach allows the system to be flexible and easily maintainable, ensuring efficient handling of user requests and responses. The RESTful architecture contributes to the application's ability to scale and maintain high performance under varying loads.

Ease of Use

The system should allow users to navigate and perform tasks efficiently, with common actions accessible within three clicks from the main dashboard. We have a simple navbar that allows the users to select different views of the application and provides functionality

available within 3 clicks. For example, going into the profile and editing the user's details.

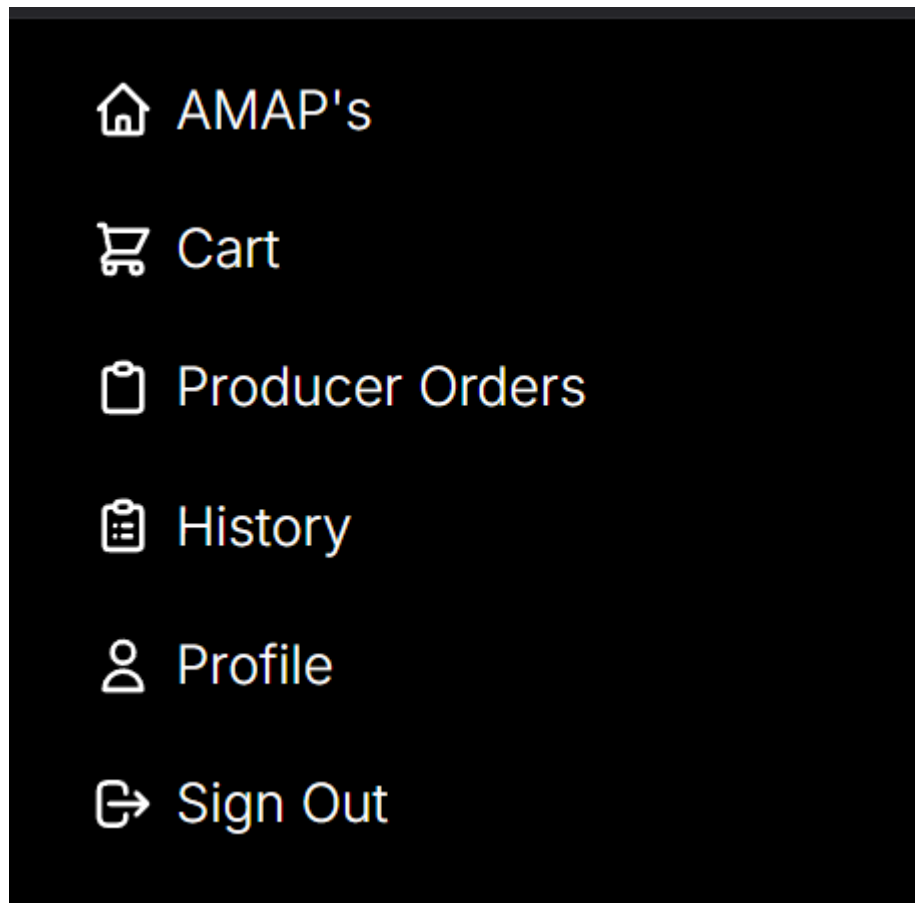


Figure 79 UI screenshot of the application's navbar

For language clarity we didn't follow the initial plan of translating the text to Portuguese, so the main language of the application does not align with the system's intended audience. Regarding consistency the terminology used is the same for the entire application, we do not mix names and values. If a user understands a product as being a simple product and a basket as being a set of simple products then that is the correct terminology, and we keep it equal for other aspects of the application.

For simplicity the text we use is very straightforward, we keep a conversational language and minimize the technical jargon, for example the way we represent the profile page. We keep it extremely simple.

All text is reviewed and has the correct grammar, spelling and contextual relevance. Our tone is professional and polite, intended for the target audience, with phrasing that conveys reliability and professionalism.

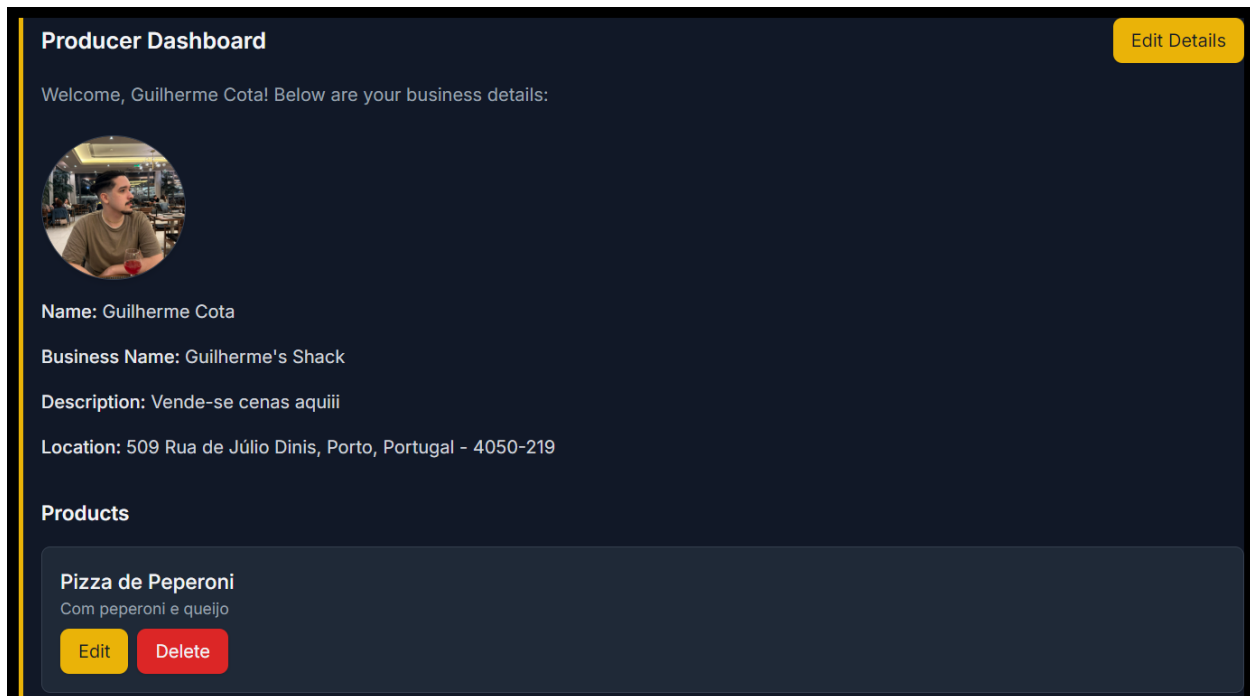


Figure 80 UI screenshot of a producer's profile page

For performance requirements, we do not have calculated metrics of performance currently. However, we can scale the project by replicating it into a new container, since we are making use of docker compose files, we can build images and open up new containers. This allows for scalability and no down time maintenance. By deciding to have a micro-services approach then we can handle these non-functional requirements a lot easier in the future.

The time-zone we are using are Western European Time, with automatic adjustment for daylight saving time in Portugal. For timestamps the format we use is YYYY-MM-DD HH:MM:SS which is the default format that supabase provides.

The currency being used is Euros, which is the standard in Portugal, we are also using numeric values representing price and payment values with floats, this allows for fractional numbering which is a lot more precise.

For GDPR compliance requirements we currently do not have setup the transparency, to inform users about what data is collected why and how. We also currently are not asking for consent for data collection, ensuring that users actively opt-in. We have incorporated data protection measures such as data minimization, ensuring that the backend only collects and store data that is strictly necessary for the application's functionality. We also restrict the amount of data sent from the client to the server, to ensure minimal exposure. For encryption, supabase ensures database-level encryption for stored user data, storing sensitive data such as JWT tokens or user credentials with encrypted passwords. For data in transit, we enforce HTTPS for all communication between the frontend, backend and supabase. We use WebSocket Secure for real-time communication via supabase subscriptions. The use of JWT Tokens, we set short lifespans for tokens and implement automatic renewal through refresh tokens, using also a robust secret key RSA for signing and verifying JWTs.

For Localization testing we haven't conducted tests to ensure that the interface elements, messages and notifications are accurately phrased and displayed without errors, especially for currency and numeric formats.